

UNIVERZITET U BEOGRADU
FAKULTET ORGANIZACIONIH NAUKA

ZAVRŠNI RAD

RAZVOJ 2D VIDEO IGRE
U GODOT ENGINE-U

Mentor:
doc. Uroš Šošević

Student:
Aleksandar Trifunović
0224/2018

Beograd, 2025. godine

Sadržaj

1. Uvod.....	5
1.1. Cilj rada i njegova struktura.....	5
1.2. Motivacija i lični cilj projekta	7
2. Definicije i pojmovi	8
2.1. Video igre.....	8
2.2. Mehanika video igre	11
2.3. Core gameplay loop.....	12
2.4. Žanr video igre	14
3. Dizajn video igre koja će biti razvijena	16
3.1. Opšte karakteristike	16
3.2. Mehanike igre razvijane u okviru rada	17
3.3. Core gameplay loop.....	18
3.4. Žanr video igre koja se razvija	20
3.4.1. City builder.....	21
3.4.2. Deck builder.....	21
3.4.3. Roguelike.....	22
4. Pregled tržišta i konkurentska analiza.....	23
4.1. Kratak opis stanja tržišta	24
4.2. Definisanje ključnih pojmova i metrika	25
4.2.1. 30-day average players (average CCU)	26
4.2.2. Recenzije igre na platformi (npr. na Steam-u).....	26
4.2.3. Prosečna ocena igre	26
4.2.4. Životni ciklus igre	26

4.3.	Identifikacija i analiza reprezentativnih igara	28
4.3.1.	City builder: Anno 1800	29
4.3.2.	Deck builder i roguelike: Slay the Spire	29
4.3.3.	City builder i roguelike: Kingdom: Two Crowns	29
4.3.4.	Uporedni pregled tržišnih metrika (Steam i SteamDB)	30
4.4.	SWOT Analiza	31
4.4.1.	SWOT tabela	32
4.4.2.	Zaključak iz SWOT analize	33
5.	Izbor tehnologije	35
5.1.	Kratko predstavljanje i poređenje dostupnih opcija	35
5.1.1.	Unity engine	35
5.1.2.	Unreal Engine	37
5.1.3.	Godot engine	38
5.2.	Izbor in-game engine-a i programskog jezika	39
5.3.	Alati korišćeni tokom razvoja	40
6.	Tehnička implementacija	42
6.1.	Arhitektura projekta	43
6.1.1.	Pregled podsistema, njihovih odgovornosti, međusobne komunikacije i načina implementacije	43
6.1.2.	Tok stanja igre	45
6.1.3.	Godot obrasci: scene, čvorovi i teme	46
6.1.4.	Determinizam i mogućnost ponavljanja	47
6.1.5.	Performanse	47
6.1.6.	Mogućnost nadograđivanja i data-driven pristup	48
6.2.	Organizacija koda i fajlova	49

6.2.1.	Struktura direktorijuma.....	49
6.2.2.	Pravila i konvencije korišćene prilikom imenovanja	51
6.2.3.	Data-driven sistem (učitavanje i validacija).....	52
6.2.4.	Razdvajanje UI-a i logike	53
6.3.	Data driven sistem	54
6.3.1.	Šeme resursa	54
6.3.2.	Primena efekata i troškova (servisni sloj).....	58
6.3.3.	Primeri instanci resursa (karata i događaja).....	60
6.3.4.	Prednosti i posledice ovakvog pristupa.....	64
7.	Rezultati i evaluacija	66
7.1.	Funkcionalni rezultati igre „Karte i kolonije“.....	67
7.2.	Tehničko ponašanje i UX	68
7.3.	Ostvarenje ciljeva dizajna	69
7.4.	Poznata ograničenja trenutne verzije.....	69
8.	Zaključak.....	71
8.1.	Naučene lekcije.....	71
8.2.	Preporuke i sledeći koraci	71
8.3.	Završna reč.....	72
9.	Literatura	73

1. Uvod

1.1. Cilj rada i njegova struktura

Cilj ovog rada je detaljno beleženje i dokumentovanje procesa razvoja 2D video igre koja je kombinacija tri popularna i dobro ustanovljena žanra: city builder, deck builder i roguelike. To će se postići simulacijom razvoja ove igre od strane malog tima u modernom okruženju i okolnostima. Kako bi rad postigao zadati cilj, on će biti podeljen na sledeće segmente, svaki od kojih ima svoj zadatak i poentu:

- Uvod
- Definicije i pojmovi
- Dizajn video igre koja će biti razvijena
- Pregled tržišta i konkurentska analiza
- Izbor tehnologije
- Tehnička implementacija
- Rezultati i evaluacija
- Zaključak
- Literatura i izvori

Uvod ima zadatak da na jasan i precizan način približi čitaocu cilj rada, njegovu strukturu, motivaciju i lični cilj pisca.

Unutar segmenta za definicije i pojmove biće uvođene određene fraze i reči koje su neophodne za dalje razumevanje i praćenje rada. One će zatim biti jasno definisane, a te definicije će zatim biti detaljnije obrađene i na kraju će biti dati primeri za njih.

Tokom dela dizajn video igre koja će biti razvijena biće navedene i određene glavne karakteristike video igre koja je centralna tema ovoga rada. One će poslužiti kao glavni kriterijumi za dalje orijentisanje i razradu ovog rada i same video igre.

U okviru pregleda tržišta i konkurentske analize detaljnije će biti obrađeno trenutno stanje tržišta video igara i na taj način će biti dokazano da je razvoj video igre zapravo unosna i opravdana investicija. Biće Predstavljeni primeri uspešnih video igara koji će poslužiti kao primer i ujedno potencijalna konkurencija igri koja se razvija u okviru rada. Na kraju poglavlja će stajati urađena SWOT analiza date igre u poređenju sa već

pomenutim primerima i na osnovu te analize biće izvedeni zaključci koji bi pomogli u daljem razvoju.

Segment izbor tehnologije će sprovesti čitaoca kroz veoma bitan proces izbora u kom game engine-u i programskom jeziku će se razvijati video igra, kao i izbore drugih alati koje pomažu prilikom ograničavanja i planiranja razvoja i kreiranja drugih asset-a koji se koriste prilikom razvoja poput modela, sprite-ova i drugog.

Dizajn igre će detaljno definisati i objasniti žanrove koji su poslužili kao inspiracija i uzor, kao i koncept core gameplay loop-a. Takođe će se detaljnije obraditi šta se tačno od kog žanra uzima i integriše u samu igru i šta se postiže time. Ovde će se i predstaviti koncipiranje i razvijanje mehanika same video igre, definisanje vizuelnog stila i pravljenje UI i UX delova. Na samom kraju će se uvesti koncept modularnog sistema i pojasniti zašto je generalno bitan ali i zašto ga i kako implementiramo u ovoj video igri.

Tehnička implementacija će demonstrirati kako su odluke donesene u segmentu za dizajn igre implementirane u realnosti uz pomoć tehnologija i sistema koje su usvojene u okviru segmenta za izbor tehnologije.

Segment za rezultate i evaluaciju za cilj ima da predstavi finalni proizvod čitaocu, šta je sve od originalnog plana uspešno implementirano a šta nije postignuto putem uporedne analize i na kraju će biti detaljan zaključak u okviru koga bi se propitao potencijal za dalji razvoj i marketibilnost ovog proizvoda.

U zaključku bi se spomenule sve naučene lekcije prilikom procesa razvoja video igre, tegobe i izazovi koji su se pojavili i prikaz rešenja koja su nađena, a potom i implementirana. Biće izloženo i konačno mišljenje pisca o celom projektu.

U literaturi i izvorima biće sistematski navedeni svi radovi koji su bili korišćeni prilikom pisanja dokumentacije ovoga rada.

1.2. Motivacija i lični cilj projekta

Ideja za izradu ovog projekta nastala je kao spoj ličnih interesovanja, prethodnog iskustva i profesionalnih ambicija. Autor teksta je od malena bio fasciniran računarom i računarskim sistemima. Brzina, efikasnost rada računara i njegova nedvosmislena logika predstavljaju jedan jasno definisan i rigidan sistem koji može da realizuje bilo koju ideju koju bi korisnik mogao da smisli pod uslovom da zna kako da je implementira. Usled navedenih karakteristika računara i računarskih sistema autoru se bavljenje njima činilo jako zanimljivo i privlačno zbog čega se i na kraju opredelio da se bavi programiranjem. Pored izbora alatki za razvoj diplomskog rada neophodno je bilo i opredeliti se i šta treba razviti. Nakon razmatranja svih mogućnosti kao i uračunavanje svih prethodnih radova koje je autor pisao i razvijao, on se opredelio za video igre.

Video igre su oduvek bile jedan od hobija autora, zbog čega su pogodna tema za diplomski rad usled dobrog poznanstva i iskustva autora sa njom. Same video igre su isto i odličan primer kreativne i efikasne upotrebe pomenutih računarskih karakteristika. Bitno je i naglasiti da su video igre jedne od težih proizvoda za razvoj i implementaciju čineći ih i ozbiljnim izazovom.

Lični cilj autora je da sprovede samoga sebe kroz proces razvoja video igre, tokom kog će konstantno testirati svoje menadžerske, analitičke i programerske sposobnosti kako bi simulirao razvoj video igre i na taj način pripremio sebe za budućnost i dalji razvoj svoje karijere. Adekvatnim vođenjem dokumentacije i pažljivim zapisivanjem svakog procesa tokom razvoja autor želi da pokaže svoje akademske sposobnosti ali i ujedno demonstrira svoj tok razmišljanja prilikom ovakvog rada.

2. Definicije i pojmovi

Kako bi primenili bilo koju naučnu metodu na datu temu neophodno je pre svega uvesti jasnu strukturu. Ovo će se postići jasnim definisanjem određenih pojmova koji će zatim biti korišćeni u formiranju te strukture. Definicije će biti uvedene pozivanjem na već napisane i prihvaćene naučne radove. Tako uvedene definicije će se zatim detaljnije obraditi i time obrazložiti zašto su usvojene, kao i šta se može zaključiti ili uvideti na osnovu njih. Pojmovi koji će se obraditi su:

- Video igre
- Mehanika video igre
- Core gameplay loop
- Žanr video igre

2.1. Video igre

U okviru ovog rada usvojena je definicija Nicolas Esposito-a koja glasi: „Video igra je igra koju igramo pomoću audio i vizuelnih aparatura i može ali i ne mora biti zasnovana na nekoj priči“. U okviru ove definicije bitno je razumeti šta sve obuhvataju određeni delovi i pojmovi ove definicije, pa ih je zato neophodno detaljnije razraditi:

- Igra – Iako je većini poznat pojam igre, ovde se stručno definiše kao dobrovoljna interaktivna aktivnost u kojoj jedan ili više igrača poštuju pravila koja im ograničavaju ponašanje. Ova ograničenja i pravila prouzrokuju stvaranje konflikta koji zatim igrači rešavaju dok se konflikt ne završi sa određenim rezultatom.
- Igramo igru – Ovde će igranje biti definisano kao slobodno kretanje i manevrisanje u sistemu koji je ograničen i definisan pravilima igre koja se dobrovoljno prihvataju i apsolutno poštuju.
- Audio i vizuelne aparature – Fundamentalna razlika između video igara i tradicionalnih igara jeste predstavljanje tih pravila i ograničenja. Tradicionalne igre moraju da vode računa o složenosti i kompleksnosti pravila kako bi svi igrači mogli da ih prate i poštuju ali i da uporedno kontrolišu ostale igrače. Video igre predstavljaju svoja pravila putem računara, što im daje znatnu prednost. Zahvaljujući računarima video igre mogu da nezavisno obračunaju i vode računa

kako i koliko se mnogo pravila poštuju što omogućava implementaciju većeg broja tih pravila ali i usložnjavanje tih pravila. Ovakav sistem omogućava automatizaciju obrade ponašanja igrača u kom video igra prvo prihvata unos od igrača, zatim na osnovu utvrđenih pravila vrši obračune i određuje posledicu tog unosa koju zatim predstavi igraču putem vizuelnih naznaka (primera radi snimka ili animacije) ili audio naznaka (recimo odigra se specijalni efekat ili kratka pesma).

- Priča – Video igre mogu a i ne moraju da sadrže narativne elemente. U slučaju da ih ne sadrže video igre su onda koncipirane kao apstraktni izazovi koje igrač treba da prevaziđe i savlada. Dobar primer toga bi bile jedne od prvih video igara: Pong i Tetris. U Pong-u ili Tetris-u igraču nisu dati nikakvi narativni elementi, već samo pravila koja treba da poštuje i cilj koji treba da postigne. Igre koje imaju narativnu komponentu mogu da se drastično razlikuju u kojoj meri je imaju. Neke igre daju minimalan ali opet prisutan narativni element poput Minesweeper-a. Putem malih crteža zastavica i bombi može se naći kratak narativni okvir o nekom entitetu koji je zadužen da na određenom prostoru pronađe sve mine kako bi se uklonile, a da pritom slučajno ne aktivira jednu. Druge video igre su zaista u potpunosti i fokusirane na priču i naraciju kako bi što bolje predstavile njihov svet. Dobar primer bi bio The last of us gde je obrađena priča starijeg muškarca u svojim poznim četrdesetim i mlade devojkice od oko 12 godina kako pokušavaju da prežive i da se snađu u post apokaliptičnom svetu gde je parazitska vrsta gljiva usmrtila znatan procenat ljudske populacije. Uprkos ovako drastično različitim pristupima ka implementaciji narativnih elemenata u video igrama moguće je povući jedan zajednički element: da je igranje same igre centralna komponenta kojoj priča potpomaže, a ne obrnuto. Priča ima zadatak da igrača dodatno uvuče u samu igru, odnosno igranje same igre.

Ako se uzmu svi ovi elementi u obzir, počinje da se polako shvata zbog čega su video igre uspele da imaju tako eksponencijalan rast popularnosti i marketibilnosti u proteklih par decenija:

- One ne opterećuju igrača van samoga igranja, s obzirom da vode računa o pravilima i posledicama akcija koje korisnik preduzima. Ovo značajno olakšava

igranje i uprošćava proces učenja same igre pošto ne zahteva od igrača da prvo nauči sva pravila pa tek onda može da igra.

- Kroz sam proces igranja, igre zahtevaju stalnu pažnju igrača, što olakšava prenos narativnog elementa. Direktnim učešćem, igrač se dodatno povezuje sa pričom u kojoj je sada postao aktivni učesnik.
- Igranje stvara utisak ličnog učešća, čime se formira emotivna veza između igrača i igre. Tako igra odaje utisak da korisnik pravi lični doprinos, što dodatno motiviše korisnika da nastavi da igra i usvaja nove informacije koje ta igra pruža.

Sve navedeno formira sistem povratne sprege između igrača i igre koja garantuje da će svaka kvalitetna igra imati veliku i odanu publiku. Ovakva povratna sprega ukazuje na sve moguće primene i potencijale video igara.

Pored tradicionalnog pristupa gde su igre proizvod na velikom tržištu, svrha se može naći i u edukativnom, kulturnom i socijalnom smislu. U edukaciji usled gejmfikacije (eng. gamification) koja tradicionalni pristup učenju menja sa novim sistemima koji podsećaju na igru i na taj način motivišu učesnike na angažovanje, istraživanje, rešavanje problema i olakšavaju proces učenja video igre su dokazale svoju korist u tom procesu.

U kulturi video igre omogućavaju učešće igrača u samoj priči, zbližavajući temu i motive te priče na do sada neostvarljiv način.

Sa socijalnog aspekta u popularnim video igrama gde više igrača može da učestvuje istovremeno zahvaljujući mehanikama i dizajnerskim rešenjima koja podstiču međusobnu interakciju igrača. Video igre služe kao platforma i baza za upoznavanje i zbližavanje ljudi, pružajući zamenu za tradicionalna mesta za okupljanje poput kafića i barova.

2.2. Mehanika video igre

Nakon uspešnog definisanja koncepta video igara, na redu je da se objasni kako one funkcionišu. Ovo će se postići uvođenjem i razradom pojma mehanika video igre. Ovaj pojam će biti od velike važnosti prilikom planiranja i implementiranja video igre koja se razvija.

U ovom radu usvojena je definicija Miguel Sicart-a koja glasi: „Mehanike video igara su metode pozvane od strane agenata koji su dizajnirani zarad interakcije sa stanjem video igre“. Svaka komponenta ove definicije će se detaljnije obrazložiti.

- Stanje video igre – ovo je kolokvijalni naziv koji treba da obuhvati celokupnu situaciju koja je predstavljena u video igri u jednom datom trenutku u vremenu. To je zapravo skup podataka i njihovih vrednosti. Primer za ovo bi bio da ima neka igra u okviru koje igrač brine o nekom selu i zarad te brige barata sa resursima. Neka se vodi da u ovoj igri ima neki resurs koji se zove hrana i da se hrana meri u tonama. Unutar igre hrana je predstavljena sa pozitivnim celim brojem (odnosno integer-om). Taj broj je zapravo deo stanja video igre.
- Interakcija sa stanjem video igre – stanje video igre može da varira u zavisnosti od trenutka posmatranja. Svaki podatak koji se uključuje u stanje video igre može a i ne mora da ima interakciju koja direktno utiče na njegovu vrednost. U konkretnom slučaju igre koja se razvija ako se ponovo upotrebi primer sa hranom to bi bila interakcija promena vrednosti hrane. U jednom trenutku vrednost hrane može da bude primera radi 5 i da predstavlja 5 tona hrane, dok u drugom može da bude 15 tona, a u trećem vrednost hrane može da bude svega 3 tone. Sam akt menjanja stanja (odnosno vrednosti podataka tog stanja) predstavlja interakciju sa tim stanjem. Ta interakcija se u okviru igara predstavlja i implementira putem metoda.
- Metode – one predstavljaju mehanizme ili ponašanja koje stoje agentima na raspolaganju i pomoću kojih ti agenti utiču na stanje video igre. U okviru igre u razvoju, ako koristimo primer sa hranom, to bi bila metoda za uvećavanje količine hrane i metoda za umanjivanje količine hrane. Takođe se može implementirati više metoda koji rade sličnu ili istu stvar ali na drugačije načine. U okviru rada bi to bila metoda za uvećavanje hrane putem žetve gde, u zavisnosti od površine njive koju je igrač posejao, se proporcionalno povećava količina hrane sa kojom igrač

raspoláže, dok bi druga metoda bila metoda uvećavanje hrane putem kupovine gde se količina hrane proporcionalno povećava sa količinom novca koju je igrač odlučio da upotrebi prilikom kupovine.

- Agenti – u okviru video igara svi učesnici koji mogu putem metoda da utiču na stanje video igre predstavljaju agente. To mogu da budu ljudi koji igraju tu igru ali mogu i da budu agenti koji su ugrađeni u samu video igru. Za igru iz ovoga rada agenti bi bili igrači dok biraju da li će i koliko hrane da kupe, ali agent bi isto bio i recimo neki stanovnik sela o kome se igrač brine koji konzumira tu hranu i na taj način je umanjuje.

Razumevanje i strukturisanje mehanike igre kroz ovu definiciju omogućava jasno planiranje logike i funkcionalnosti sistema. Takođe, predstavlja osnovu za dizajniranje core gameplay loop-a i pravilno klasifikovanje igre u odgovarajući žanr.

2.3. Core gameplay loop

Sledeći pojam koji je bitan zarad daljeg definisanja, projektovanja i implementiranja video igre je core gameplay loop. On će pomoći da se koncipiraju i detaljnije obrade interakcije između igrača i same igre koje proizilaze iz mehanika igre.

Ovaj rad će se pozivati na tumačenja Emmanuel-a Guardiola koji prvobitno uvodi definiciju za gameplay da je: „Skup svih radnji koje igrač preduzima, a koje mogu pozitivno ili negativno da utiču na trenutno neizvesnu situaciju u kojoj se on nalazi unutar igre“. Fokus unutar ove definicije se stavlja na skup svih radnji, odnosno akcija. Te radnje je podelio na dva dela:

- In-game actions – radnje unutar same igre. Ovo su konkretne akcije koje igrač preduzima i imaju direktne i konkretne posledice na stanje video igre. Ako se malo bolje pogleda može se uvideti da su ovo zapravo ništa drugo nego mehanike igre koje je igrač pozvao u tom trenutku. Ako bi koristili dosadašnji primer ovo bi bio trenutak kada igrač klikće dugme da putem trgovine uveća količinu hrane sa kojom raspoláže.
- Out game-actions – akcije koje igrač obavlja ali ne uključuju direktan unos u igru, ali utiču na tok samoga igranja. U okviru primera hrane to bi bilo kada bi igrač aktivno debatovao sa sobom koliko hrane da kupi prilikom trgovine. Iako samo

razmišljanje nije predstavljeno unutar igre, konačan rezultat te aktivnosti će uticati na količinu hrane koju će igrač da kupi.

Unutar nastavka teksta Emmanuel uvodi i koncept petlje (loop). On smatra da kombinacijom sistema izazova, nagradama i ograničenjima koja su postavljena unutar igre nastaje petlja aktivnosti koje igrač konstantno izvodi:

- identifikovanje problema ili izazova unutar igre
- donošenje odluke
- izvršenje akcije
- evaluacije rezultata

Bitno je i navesti da svaka nova iteracija kroz koju igrač prolazi se obično nadovezuje na posledice prethodne. Ovo predstavlja i cilj samog gameplay loop-a: da kreira izazove koji angažuju i stimulišu igrača i da omoguće kroz tu repeticiju učenje pravila. Učenje omogućava igraču da se vremenom poboljša i na taj način ga dodatno podstiče da nastavi igranje igre. To konstantno poboljšanje igrača treba pratiti sa postepenim otežavanjem igre putem podizanja zahteva ili usložnjavanjem ove petlje preko dodavanja novih koraka ili povećanjem broja potencijalnih rešenja prilikom donošenja odluke.

Pojam 'core' unutar fraze core gameplay loop naglašava da je ova specifična petlja sam srž igre i da će se oko nje cela igra vrteti. U njoj će igrač provesti najviše vremena, biće najbolje upoznat sa njenom problematikom i potencijalnim rešenjima i ona će biti glavni razlog uspeha ili neuspeha igre.

Emmanuel navodi da core gameplay loop pomaže prilikom razvoja igre da se kreira model ponašanja igrača koji se zatim koristi u sledećim fazama:

- dizajn – pomaže prilikom koncipiranja osnovnog iskustva koje igra želi da ponudi igraču
- produkcija – unutar produkcije služi kao osnova za balansiranje težine i dinamike igre
- evaluacija – omogućava preciznu analizu učestalosti i značaja pojedinih radnji

U okviru ovog rada ovaj koncept će se javljati i biće detaljnije obrađen unutar pomenutih faza.

2.4. Žanr video igre

Zarad dalje obrade podataka, pogotovo podataka za tržište video igara, neophodno je uvesti pojam žanra video igara. Kao i u drugim medijima žanr je pogodno tlo za kategorizaciju i grupisanje sličnih proizvoda radi lakšeg poređenja i rangiranja. Međutim definisanje žanra je znatno teži i zahtevniji poduhvat u odnosu na prethodne pojmove. Za razliku od video igara, core gameplay loop-a i mehanike video igara kod kojih više naučnika i naučnih radova imaju različite definicije čije su suštine i srži iste, kod žanrova imamo drugačiju priču.

Ovaj problem je Wolf, M. J. P. prvi pokušao da razreši 2001. godine kada je u svome radu pokušao da mapira taksonomiju žanrova po mehanikama i perspektivi igranja. Kasniji autori, poput Apperley-a i Arsenault-a, ističu da pristup Marka Wolfa nije bio dovoljan za obuhvatnu analizu žanrova. Oba autora na različite načine skreću pažnju na kriterijume koje je Wolf koristio: mehanike video igara i perspektivi igranja. Arsenault tvrdi da se žanrovi ne baziraju isključivo na formalnim osobinama i karakteristikama igre (poput grafike, perspektive i mehanike), već i na očekivanjima samih igrača i širem kontekstu. Apperley u svome radu dolazi do vrlo sličnog zaključka. Po njemu žanr video igara se ni ne može definisati kao stabilna kategorija, već kao „performativna praksa“ koja zavisi od tehnoloških, kulturnih i društvenih faktora. Može se uočiti da su se u akademskim krugovima za definisanje žanrova video igara formirale dve struje sa različitim stavovima: jedan tvrdi da problematici se treba pristupiti naučnim i logičkim putem, služeći se isključivo formalnim karakteristikama igara, dok drugi tvrdi da se treba pristupiti sa socijalnog aspekta, odnosno stavova, mišljenja i tumačenja igrača.

U ovome radu biće primenjen dvojaki pristup koji bi trebao da zadovolji obe strane. Na ovaj način će biti omogućena detaljnija analiza igre i kroz perspektivu korisničkog doživljaja, ali i kroz strukturalnu analizu njihovih mehanika. Ovako dvojaki pristup će pomoći da se adekvatno pozicionira igra na tržištu kada je njen razvoj završen, ali i ujedno da se planira što bolji i efikasniji način izrade igre.

Prvi pristup će biti zasnovan na samom tumačenju žanrova na osnovu doživljaja, očekivanja i iskustva koje igrači povezuju sa određenim žanrovima. Ovaj pristup će biti značajan za obradu podataka vezano za samo tržište, s obzirom da cilj ovog rada je da

kreira proizvod koji će biti što uspješniji, a samim tim su mišljene i preferencije ciljne publike (odnosno igrača) najbitniji podaci i kriterijumi.

Drugi pristup će kreirati opšte grupacije video igara na osnovu njihovih mehanika. Ovakav tip raspodele će biti značajan prilikom poređenja načina razvoja video igara. Na ovaj način će moći lakše da se porede različiti alati, pristupi problematici i generalni način izrade i razvoja video igre i da se onda odabere najprikladniji pristup koji bi najviše odgovarao prilikom izrade igre u okviru ovoga rada.

3. Dizajn video igre koja će biti razvijena

U ovom segmentu rada će biti objašnjena i definisana igra koja se pravi u okviru rada pomoću ranije pojašnjenih pojmova. Biće navedene i obrazložene opšte karakteristike igre, zatim na osnovu njih će se definisati mehanike igre i formulisati core gameplay loop. Na samom kraju će ova igra biti klasifikovana u odgovarajuće žanrove na osnovu dva ranije napomenuta kriterijuma. Ovo se sve radi kako bi snalaženje prilikom procesa planiranja i dizajniranja igre bilo znatno olakšano, kao i prilikom obrade podataka stanja tržišta.

3.1. Opšte karakteristike

Igra čiji razvoj će biti dokumentovan u okviru ovog rada se zove karte i kolonije. Odvija se okvirno u 15. veku prilikom pronalaska novog omanjeg ostrva. Igrač (koji je i ujedno jedini igrač) je postavljen u poziciju kolonijalnog guvernera koji za cilj ima da osnuje, a zatim upravlja naseljem na tom ostrvu tokom svog mandata. Cilj postavljen igraču je da adekvatnom upotrebom ograničenog broja resursa sa kojim raspolaže obezbedi sigurnost i prosperitet naselja nad kojim upravlja u ograničenom broju poteza. Prosperitet je prikazan brojem i predstavlja kvantifikovani uspeh koji je postigao sam igrač. Broj poteza u okviru igre predstavlja dužinu mandata koji mu je dodeljen i ograničen je. Ovo je urađeno kako bi se napravio izazov da se poveća prosperitet što je više moguće tokom tog mandata. Prepreke i izazovi koji bi igraču otežavali postizanje tog cilja bili bi razni privremeni događaji koji bi se periodično pojavljivali i zahtevali da ih igrač nekako razreši (na primer suša, pandemija, otkrivanje novog resursa na ostrvu itd.).

Postoje mnoge igre koje imaju slične ili iste karakteristike sa igrom koja je obrađivana u ovome radu te je neophodno uvesti neku jedinstvenu karakteristiku po kojoj bi ona bila prepoznatljiva. Za igru razvijenu u okviru ovoga rada to će biti da mogućnosti sa kojima igrač raspolaže su predstavljene putem karata. Ideja je da se ograniči igrač tako da može da odigra samo ono što trenutno ima u ruci. Na taj način se dodatno povećava izazov za igrača, čineći igru zanimljivijom. Igrač može da utiče na sam špil karata sa kojima raspolaže tako što gradi i usložnjava svoje naselje i da tako doda nove ili poveća broj već postojećih karata.

Na primer, igraču su potrebni resursi za gradnju. Igrač razgleda koji su mu svi resursi dostupni na ostrvu. Na ostrvu postoje razna nalazišta resursa koja određuju šta od resursa može da se sakuplja i koliko mnogo. Primetio je da na svom ostrvu ima velika šuma bogata visokokvalitetnim drvećem. Ona omogućava sakupljanje građevinskih resursa, te odigrava kartu da se izgradi kamp za drvoseče. Kamp daje konstantan priliv resursa za izgradnju, ali ujedno dodaje karte za povećavanje proizvodnje u špil, kao i kartu za unapređenje kampa za drvoseče u esnaf drvoseča. Ako igrač kasnije odluči da odigra kartu za povećanje proizvodnje, priliv resursa za izgradnju će se privremeno povećati ali će onda postojati i rizik da drvoseče previše drveća poseku, smanjujući šumu i umanjujući do kog nivoa zgrada za prikupljanje drveta može da se unapredi. U slučaju da igrač odigra kartu da unapredi kamp u esnafsko udruženje drvoseča, ovog rizika preterane seče nema, ali zato da bi se ova karta uopšte i mogla odigrati, neophodno je utrošiti određenu količinu resursa.

Ovaj primer predstavlja samu srž i suštinu igre koja se razvija: upravljanje rizikom i resursima. Igra konstantno postavlja igrača pred izbor: brz razvoj sa dugoročnim posledicama ili spori rast bez rizika. Upravo taj balans između napretka i rizika čini osnovnu težinu igre.

3.2. Mehanike igre razvijane u okviru rada

Na osnovu prethodnog dela mogu se izvesti tri ključne mehanike igre koji će da budu noseći elementi:

- Mehanika posvećena baratanju resursa
- Mehanika posvećena izgradnji špila
- Mehanika posvećena razvoju naselja

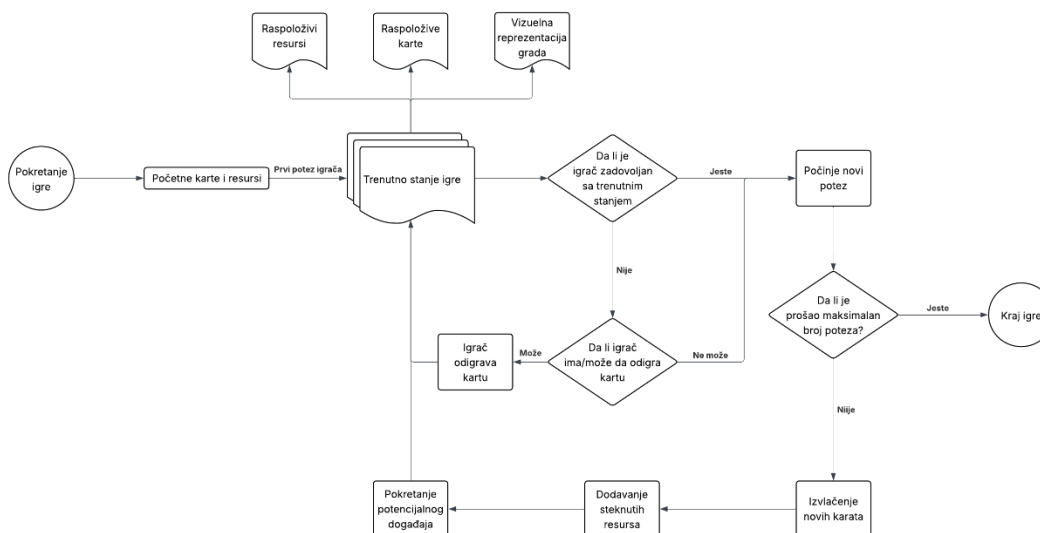
Mehanika baratanja resursa kao glavni cilj ima da kreira sistem i strukturu vrednovanja stvari u okviru igre. Sve što može da se dogodi unutar igre biće izraženo preko resursa, bilo to direktno, poput troška da se odigra karta ili događaja koji bi smanjio resurs na skladištu, ili indirektno, u vidu oportunitetnog troška. Ovo bi omogućilo igraču da lakše poredi izbore sa kojim raspolaže u datom trenutku i da brže i bolje donese odluku. Mimo vrednovanja, mehanika resursa bi isto bila i izazov koji igrač treba da prevaziđe. Igrač mora da nauči kada i gde da upotrebi resurse sa kojima raspolaže na što bolji način.

Mehanika izgradnje špila je noseća mehanika ove igre u razvoju. Ona je mehanika koja direktno i indirektno interaguje sa ostalim delovima igre. Preko nje igrač zapravo utiče na stanje igre. Za cilj ima da kreira sistem poteza sa kojima igrač raspolaže. Ono što je unikatno kod nje jeste da je taj sistem fleksibilan i dozvoljava igraču da utiče na njega tako što svojim aktovima postepeno dodaje nove karte u špil, povećavajući njihov broj ili povećavajući efikasnost već postojećih karata.

Poslednja bitna mehanika je mehanika razvoja naselja. Ona za cilj ima da vizuelno predstavi igraču trenutno stanje igre, da li sve ide kako treba ili je nešto krenulo po zlu. U okviru naselja će se nalaziti vizuelne reprezentacije odigranih i trenutno aktivnih karata. Na primer ako se ponovo upotrebi primer kamp drvoseča on bi bio predstavljen kao kolekcija brvnara unutar naselja. Normalne okolnosti rada bi bile predstavljene kao animacija drvoseča koji seku obližnje drvo. U slučaju odstupanja normalnosti, na primer hara neka pandemija koja za posledicu ima da sprečava rad izgrađenih zgrada, situacija bi bila predstavljena korisniku sa odsustvom napomenute animacije. Ako korisnik odigra kartu da unapredi kamp u esnafsko udruženje to bi isto bilo vizuelno predstavljeno korisniku tako što bi zamenili vizuelnu reprezentaciju kampa sa esnafom, jednom većom zgradom od kamena i drveta. Animacija za rad esnafa bi se takođe promenila tako što bi povećali broj zaposlenih koji sada ne što seku drvo nego ga ujedno i unose u zgradu za dalju obradu kako bi se adekvatno odrazilo povećanje proizvodnje.

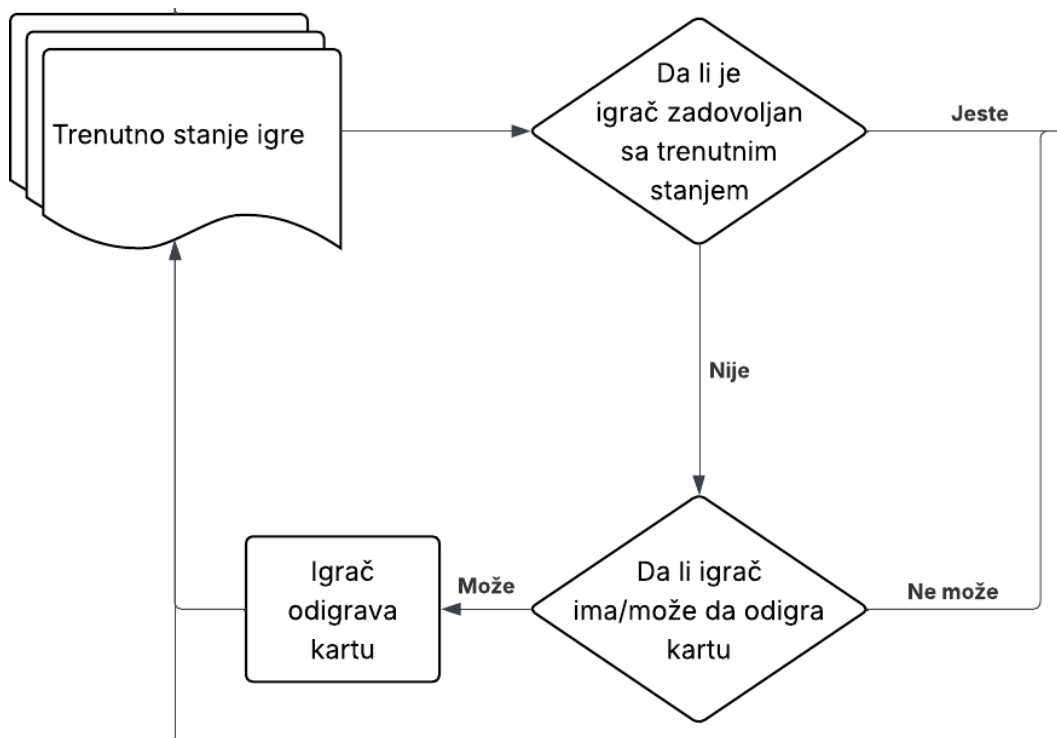
3.3. Core gameplay loop

Od velikog značaja je da se definiše međusoban odnos mehanika i igrača radi lakšeg planiranja i projektovanja razvoja. Mimo planiranja, core gameplay loop treba slikovito da prikaže šta se tačno omogućava igraču da radi i na koji način je dozvoljeno da igrač utiče na igru, ali ujedno i kako da igra, i na koji način, utiče na igrača. S toga, najbolje je da se prvobitno gameplay loop prikaže vizuelno u vidu dijagrama koji će se zatim dodatno obrazložiti i pojasniti.



Na dijagramu je predstavljen celokupan proces igranja jedne partije ove video igre u razvoju. Prilikom pokretanja partije igrač ima prvobitno početnički špil koji služi kao osnova kao i mali broj resursa. Sve napomenuto zajedno sa stanjem ostrva i naselja predstavlja trenutno stanje igre. Igrač prvobitno određuje da li je zadovoljan sa trenutnim stanjem. U slučaju da nije, igrač može da ga promeni odigravanjem neke karte koja mu se trenutno nalazi u ruci. Prilikom izbora karte igrač mora da uračuna resurse koje trenutno ima kao i potencijalnu situaciju u kojoj se nalazi naselje. Mora da razmišlja unapred i da donese odluku koja je po njemu najoptimalnija. Kada igrač ne može više karata da odigra ili kada je zadovoljan sa trenutnim stanjem igre, on završava potez. Po završetku poteza, sistem igre preuzima dalje procese. Pre svega proverava se da li se maksimalan broj poteza postigao. U slučaju da jeste, igra se završava, u suprotnom, igra se nastavlja tako što se iz špila izvlače nove karte, zatim se u zalihe dodaju novostečeni resursi i na kraju se pokreće proces događaja. U procesu događaja nasumice se bira neki događaj koji utiče na stanje igre u tekućem potezu. Ti događaji mogu i pozitivno i negativno da utiču na stanje (na primer usled obimnih padavina igrač je dobio više hrane u ovome potezu ili usled manjka padavina igrač je dobio manje nego sto je očekivao).

Iz definisanog gameplay loop-a može da se izvuče konkretan deo u kome bi bilo poželjno i na koji bi se ciljao da se zadrži igrač, odnosno core gameplay loop:



U ovome delu igre igrač ima maksimalnu kontrolu i uticaj na stanje igre. Unutar nje će igrač donositi bitne odluke koje će imati posledice na stanje igre a samim tim i njegove buduće odluke. Zbog toga sa velikim stepenom sigurnosti može se reći da je ovo najbitniji deo igre koji treba najbolje razraditi i što bolje napraviti, a ostale delove igre treba napraviti i dizajnirati tako da potpomažu core gameplay loop-u.

3.4. Žanr video igre koja se razvija

Kao što je već napomenuto postoje dva različita pristupa dodeljivanju žanrova video igri: jedan pristup želi da objektivno razdeli igre u jasno definisane žanrove na osnovu mehanika koje su implementirane u okviru igre dok drugi pristup polazi od subjektivnog tumačenja samog igrača tvrdeći da nije bitno šta ili kako je nešto implementirano unutar igre, već kako tu igru sami igrači i šire tržište doživljava i tumači. U okviru određenih konteksta ponekad bi prvi pristup imao više smisla od drugog i obratnu, te će se koristiti oba pristupa u okviru ovog rada. Prvi objektivni pristup putem mehanika će biti od koristi kako bi našli slične igre koje će se koristiti za uzor prilikom dizajniranja i implementacije igre koja se razvija. Drugi subjektivni pristup preko mišljenja igrača i opšteg javnog

mnjenja će se koristiti za pregled tržišta i da se pronađu dovoljno slične igre koje bi se kasnije koristile u okviru SWOT analize i za predikcije potencijala igre koja se razvija.

Kombinacijom i mehaničkog i subjektivno-perceptivnog pristupa odrediće se prvo žanrovi u koje bi klasifikovali igru u razvoju, a zatim će se odabrati reprezentativne igre iz tih žanrova koje su opšte poznate i popularne zarad poređenja sa igrom iz ovoga rada. Ovakvim pristupom su određeni sledeći žanrovi, a o primerima će biti više u nastavku teksta.

- City builder
- Deck builder
- Roguelike

3.4.1. City builder

Igra koja se pravi i analizira u okviru ovoga rada je city builder iz razloga što bi igrač razvijao naselje koja je kolonija. Igrač će postepeno da dodaje nove i unapređuje već postojeće zgrade. Baratanje resursima je takođe vrlo česta mehanika koja se implementira u igrama ovog žanra zarad unošenja kompleksnosti i izazova. Igra koja bi poslužila kao uzor iz ovog žanra bi bila Anno 1800. Ona ima veoma sličan koncept igri iz ovoga rada, gde je igrač zadužen da izgradnjom, širenjem i održavanjem više naselja na jednom arhipelagu u periodu rane industrijske revolucije. Glavni izazov se sastoji u pravljenju lanca snabdevanja resursa koji se koriste unutar igre. Zbog ovih razloga smatram da je Anno 1800 pogodna igra za poređenje i korišćenje kao uzora u ovome žanru.

3.4.2. Deck builder

Deck builder je žanr u okviru koga se može naći šarenoliki izbor različitih pristupa i implementacija ove mehanike ali suštinski je neophodno postojanje špila sa kojim igrač raspolaže i koji se koristi tokom igre. Igrač može a i ne mora da utiče na sadržaj špila. U slučaju da može da utiče na sadržaj, to radi ili u prvobitnoj fazi pripreme pred početak aktivnog dela igre ili baš u samom aktivnom delu svojim postupcima. Igra iz ovog rada bi implementirala ovaj drugi pristup, gde igrač unapređivanjem ili izgradnjom zgrada dodaje karte u svoj špil sa kojim igra, te je smisleno i klasifikovati igru iz ovog rada u ovaj žanr. Igra koju bi koristili za poređenje i uzor u okviru ovog žanra bi bila Slay the Spire. Ona je

igra koja takođe omogućava igraču da postepeno dodaje i gradi svoj špil tokom aktivnog dela igranja same igre.

3.4.3. Roguelike

Roguelike žanr je najrigidniji žanr od data tri navedena. Iako igra iz rada sadrži neke od mehanika koje su ključne za roguelike igre poput proceduralne generacije određenih elemenata igre, permadeath-a i baratanja sa resursima ipak ne bi mogla da se klasifikuje u najkasičnijem smislu. S toga igra koja se razvija bi potpala u podžanr roguelike igara, takozvane roguelite igre koje kombinuju elemente roguelike igara sa drugim žanrovima. Igru iz ovog žanra koja je preuzeta za poređenje bi bila Kingdom: Two Crowns. U pitanju je igra u kojoj je igrač kralj razrušenog kraljevstva i sada mora da istražuje proceduralno generisani svet kako bi pronašao resurse koje onda treba preraspodeliti adekvatno između obnavljanja kraljevstva i izgradnje novih odbrambenih pozicija koje se koriste da se brani to kraljevstvo tokom noći kada ga napadaju razna čudovišta.

4. Pregled tržišta i konkurentska analiza

Zarad boljeg razumevanja pozicije koju bi igra u razvoju mogla da zauzme, neophodno je analizirati trenutno stanje u industriji, postojeće konkurente i faktore koji utiču na uspešnost sličnih naslova. Ovaj segment rada ima za cilj da pruža sveobuhvatan pregled relevantnih tržišnih pokazatelja i trendova, kao i da identifikuje prilike i pretnje koje bi mogle da utiču na razvoj i komercijalni potencijal ove igre. Do sada su se u okviru rada uveli pojmovi iz industrije video igara, bili su obrazloženi i onda pomoću njih se uspešno definisao koncept igre iz ovog rada. Ovako konkretno definisane stavke igre koja se razvija u okviru rada omogućavaju da se nađu slični proizvodi koji već postoje i da se uporede sa igrom iz rada. Ovo se radi iz dvojakog razloga:

- kako bi se pronašle potencijalno dobre ideje i inspiracije u sličnim igrama koje bi rado usvojili u igru iz rada
- pronalaskom i analiziranjem uspeha i popularnosti sličnih igara probalo bi se da se projektuje potencijalan tržišni performans igre iz rada.

Metodološki pristup pronalaska sličnih video igara i njihovog poređenja sa igrom iz ovog rada će biti dvostruk. Koristiće se kvantitativne metrike koje su industrijski standard (poput prosečnog broja aktivnih igrača, broj i ocena recenzija, prisutnosti igre na drugim platformama itd.). Takođe će se upotrebiti i kvalitativni uvidi u percepciji igara od strane zajednice i stručnih kritičara kako bi se uveo širi kontekst. U skladu sa ovakvim pristupom, ovaj segment će imati sledeću strukturu:

- Kratak opis stanja tržišta
- Definisanje ključnih pojmova i metrika
- Identifikacija i analiza reprezentativnih igara
- SWOT analiza

4.1. Kratak opis stanja tržišta

U proteklih par godina industrija i tržište video igara se pokazala kao jedna od najbrže rastućih i najprofitabilnijih industrija zabave u svetu. Prema određenim analitičkim kompanijama tržište video igara je tokom 2024. godine imalo vrednost od oko 200 milijardi dolara. Dalje analize i obrade podataka su ukazale da će se trend rasta i razvoja ovog tržišta nastaviti u narednim godinama. U okviru industrije posebno veliki rast je video takozvani indie sektor, video igre koje su razvijene i distribuirane od strane malih timova ili čak pojedinaca. Razlog rasta ovog sektora može se naći u lakoći digitalne prodaje i distribucije ovih video igara.

Digitalne platforme Steam, Epic games Store, itch.io, Nintendo eShop, Playstation shop, Xbox store i mnoge druge koje posluju isključivo putem interneta u potpunosti su eliminisale potrebu za tradicionalnim izdavačima i distributerima fizičkih kopija, što je značajno smanjilo prepreke za ulazak na tržište. Digitalna distribucija je takođe i omogućila svojom infrastrukturom mogućnost konstantnog održavanja, nadograđivanja igre i novih oblika i modela monetizacije. Pored prodaje same igre, što obično i čini najveći segment prihoda, igra se može monetizovati:

- DLC-evima - prodajom dodatnih ekspanzija i proširenja osnovne video igre (Downloadable Content - DLC),
- mikrotransakcijama - unutar igre kupac zameni prave monete za digitalne koje može da troši isključivo u okviru igre,
- sezonskim propusnicama – korisnik unapred plati za određene nagrade unutar igre koje ne dobija odmah po kupovini nego ih dobija postepeno tokom igranja igre u određenom vremenskom periodu (sezoni)
- pretplatničkim modelima- postoji dva tipa pretplatničkih modela:
 - Early access model - igra se prodaje pre zvaničnog izlaska, a kupci često dobijaju bonuse (npr. rani pristup, ekskluzivne stavke, popuste na budući sadržaj).
 - Subscription pass model - igra se plasira putem servisa (npr. Xbox Game Pass), a korisnicima se omogućava pristup velikom broju

naslova uz mesečnu pretplatu, dok developer dobija deo prihoda proporcionalno broju preuzimanja i igranja.

Ovakav tip prodaje, monetizacije i distribucije je koliko prednost toliko i opasnost, s obzirom da samo na Steam-u se na dnevnom nivou objavljuju oko 30-ak video igara, odajući utisak da je tržište izuzetno konkurentno okruženje koje se često opisuje kao crveno okeansko tržište.

Uprkos velikoj konkurenciji, male igre konstantno uspevaju tako što pronadu i zatim se fokusiraju na svoju nišu. Ovo uspevaju tako što:

- pronadu i posvete se žanru sa lojalnom publikom
- imaju izuzetno jasan identitet i upečatljiv vizuelni a i generalni stil
- pružaju jedinstven i zarazan gameplay loop svojim igračima

Upravo zbog ovakvog stanja tržištu može se sa velikim stepenom pouzdanosti reći da igra „Karte i kolonije“ ima potencijala da se probije i uspe na tržištu. Kombinacijom tri žanra sa lojalnom i stabilnom publikom igra „Karte i kolonije“ ima potencijala za velikom korisničkom bazom što bi joj omogućilo da pronađe svoje mesto u indie segmentu tržišta. Promovisanjem igre putem društvenih mreža i u okviru online prodavnica i platformi čini ovu industriju dostupniju nego ikad za bilo koga sa dobrim stilom, planom i solidnim proizvodom.

4.2. Definisanje ključnih pojmova i metrika

Kako bi rad kvalitetno analizirao tržište video igara i uspeh sličnih naslova, neophodno je uvesti i jasno definisati određene pojmove i metrike koje predstavljaju standarde u evaluaciji uspeha i tržišnog potencijala video igara. Ovakvi pokazatelji i metrike omogućavaju direktno i objektivno poređenje i između igara koje nisu slične ili ne pripadaju istom žanru, a takođe omogućavaju i formiranje realnih očekivanja i racionalnih konsenzusa. Sledi navođenje i objašnjavanje metrika kojima će se rad služiti u nastavku teksta:

4.2.1. 30-day average players (average CCU)

Ova metrika predstavlja prosečan broj aktivnih korisnika koji igraju igru tokom perioda od 30 dana. Smatra se jednim od najvažnijih pokazatelja dugoročne popularnosti i održivosti igre. Ova metrika je ključna zato što velika vrednost ove metrike ukazuje na snažnu i konstantnu bazu korisnika, što može opravdati razvoj dodatnog sadržaja ili nastavka date igre. Takođe se ova metrika često koristi i prilikom evaluacije igre kod investitora ili izdavača.

4.2.2. Recenzije igre na platformi (npr. na Steam-u)

Recenzije od strane igrača često se koriste kao neformalni indikator prodajnog uspeha i zadovoljstva igrača sa video igrom. Ovo je bitan indikator zbog toga što veliki broj visokih recenzija često ima korelaciju sa visokim brojem prodatih kopija. Pozitivne recenzije povećavaju verovatnoću da će novi igrači kupiti igru, dok negativne igre mogu drastično uticati na reputaciju proizvoda, pa čak i proizvođača i samim tim na njegovu dalju prodaju.

4.2.3. Prosečna ocena igre

Ova vrednost predstavlja zbirnu ocenu koju korisnici i/ili profesionalci dodeljuju igri, najčešće u rasponu od 1 do 10. Izuzetak je Steam gde se ostavlja ili pozitivna ili negativna recenzija, a platforma zatim izračuna procentualnu zastupljenost pozitivnih recenzija u odnosu sa ukupnim brojem recenzija i zatim prikaže taj procentualni udeo. Ovaj parametar je bitan iz istih razloga kao i prethodni: može drastično da utiče na odluku da li potencijalni igrači da kupe ili ne kupe igru.

4.2.4. Životni ciklus igre

Pod ovim pojmom podrazumevamo koliko dugo igra ostaje aktivna na tržištu, odnosno, koliko dugo uspeva da zadrži pažnju i interesovanje igrača. Dug životni ciklus ukazuje da igra ima aktivnu zajednicu i zainteresovanje na duži vremenski period, što ukazuje na potencijal za dodatni sadržaj.

4.2.5. Prisustvo i uspeh na više platformi (PC, konzole, mobilni uređaji)

Prisustvo igre na više platformi i online prodavnica povećava pristupačnost i samim tim potencijalnu publiku. Dobrom adaptacijom na druge platforme povećavamo skalabilnost kao i povratak prihoda u odnosu na originalno investiran kapital (ROI – return on investment)

4.2.6. Zastupljenost i aktivnost zajednice

Ova metrika se tiče količine i kvaliteta interakcije između igrača. Zajednice igrača se grade na i putem foruma, raznih servera, zajednice za kreiranje, igranje i širenje modova i socijalnih medija (poput Youtube-a i Twitch-a). Jaka zajednica produžava život igre, podstiče i kreira organsku promociju i daje konstantan priliv informacija koje bi bile od velike važnosti za timove zadužene za razvoj i održavanje igre. Veličina zajednice je često i odličan indikator zainteresovanosti i potencijala za nastavak igre.

4.2.7. Cena igre i poslovni model

Sistem monetizacije video igre direktno utiče na očekivani prihod, ali ujedno i na percepciju igrača. On je takođe od izuzetnog značaja u kontekstu tržišnog pozicioniranja i poređenja sa konkurentskim naslovima.

Sve ove metrike i pojmovi će poslužiti kao osnov za dalje planiranje i upoređivanje igre iz ovog rada sa reprezentativnim naslovima u sličnim ili istim žanrovima. Njihovo pravilno tumačenje omogućuje stvarno sagledavanje tržišnog potencijala kao i da se na osnovu tuđih primera prepoznaju i izbegnu potencijalne greške prilikom razvoja.

4.3. Identifikacija i analiza reprezentativnih igara

U ovom segmentu će se predstaviti i analizirati nekoliko igara koje su primeri za navedene žanrove i uzori za igru „Karte i kolonije“. Cilj ove analize će biti da:

- sagleda mehanike i dizajn odluke koje su dovele do njihovog uspeha,
- primeni relevantne tržišne metrike na te naslove,
- izvuče zaključke koji će se koristiti u razvoju i pozicioniranju igre iz rada.

Kao što je prethodno definisano igra iz rada se može svrstati u sledeće žanrove:

- City builder
- Deck builder
- Roguelike

Iz svakog žanra biće predstavljena po jedna igra koja je visoko ocenjena, ima lojalnu i stabilnu korisničku bazu, prepoznatljiv stil i može da se po različitim aspektima i kriterijumima poredi sa igrom „Karte i kolonije“. Usred složenosti podataka i manjka pouzdanosti izvora tih podataka u okviru rada odlučeno je da se koriste isključivo podaci vezaie za jednu platformu – Steam.

Steam je ubedljivo najpopularnija online PC prodavnica video igara u okviru koje korisnici akumuliraju biblioteku video igara i pomoću nje preko neta skidaju igre iz svojih biblioteka na svoj računar kako bi igrali. Steam je izuzetno otvoren i transparentan, nudeći sve podatke koji su potrebni za dalju analizu na sajtu po imenu SteamDB (Steam DataBase). SteamDB je nezvanični agregator Steam podataka koji je baziran na javnim Steam interfejsima i API-evima. Zbog toga se smatra pouzdanim izvorom podataka u industriji, te će rad da se poziva na njega i da ga koristi za analizu iz istih razloga. Bitno je napomenuti i da neke igre koje će biti analizirane se prodaju i na drugim online prodavnicama (poput Epic store-a), pa čak i na drugim konzolama (na primer na Xbox-u ili Playstation-u) ali zbog manjka pouzdanih i preciznih informacija odlučeno je da u ovoj konkretnoj analizi se isključivo ograniče podaci na Steam, a za igre koje su dostupne na više mesta će biti naglašena ta mogućnost (Svi podaci su preuzeti 06.11.2025.).

4.3.1. City builder: Anno 1800

Anno 1800 je kompleksna strateška igra koja simulira ekonomski razvoj kolonija u periodu industrijalizacije. Fokus igre i njenog gameplay loop-a je na izgradnji naselja, optimizaciji prikupljanja i upotrebe resursa i trgovačkih lanaca. Igra je izašla 16. aprila 2019. godine

Relevantnost za ovaj rad:

- Bavi se kolonijalnim okruženjem isto kao i igra „Karte i kolonije“
- Fokus je stavljen na resurse i njihove lance proizvodnje i potrošnje
- Koristi složene mehanike napredovanja preko infrastrukture

4.3.2. Deck builder i roguelike: Slay the Spire

Slay the Spire je igra koja je nastala kombinacijom izgradnje špila i roguelike strukture koja se manifestovala u vidu konstantnih i uzastopnih borbi i taktičkog menadžerisanja resursa i karata. Gameplay loop igre aktivno podstiče eksperimentisanje zarad pronalaženja različitih sinergija između karata. Igra je izašla 23. januara 2019. godine.

Relevantnost za ovaj rad:

- Uzor za modularni deck building sistem
- Kratke sesije i šarenolike sinergije između karata podstiču na ponovno igranje
- Balans rizika i nagrade

4.3.3. City builder i roguelike: Kingdom: Two Crowns

Kingdom: Two Crowns je dvodimenzionalna city building igra koja implementira roguelike elemente tako što igrač ide po proceduralno generisanim nivoima i prikuplja željene resurse koje zatim koristi zarad izgradnje i unapređenja svoga grada kako bi taj grad mogao da se odbrani od napada protivnika tokom noći. Igra je izašla 11. decembra 2018. godine

Relevantnost za ovaj rad:

- Jednostavne kontrole i nizak prag ulaza
- Atmosfera i progres je zasnovan na malim i jasnim odlukama
- Sistem resursa tera igrača da razmišlja unapred

4.3.4. Uporedni pregled tržišnih metrika (Steam i SteamDB)

Metrika (SteamDB)	Anno 1800	Slay the Spire	Kingdom: Two Crowns
Datum izlaska (Steam)	16.04.2019.	23.01.2019.	11.12.2018.
Cena (EUR)	59,99 €	22,99 €	19,99 €
Ocene – All-time (procenat pozitivnih)	79,74%; ~28k recenzija	96,53%; ~191k recenzija	88,46%; ~36k recenzija
30-day average (CCU)	4.917	12.102	2.975
All-time peak (CCU)	25.302 (16.04.2019)	33.086 (18.02.2018)	26.623 (13.10.2024)
24h peak (CCU)	4.480	17.558	1.948
Indeks aktuelnosti (24h/all-time)	0.177	0.531	0.073

Indeks aktuelnosti se dobije kada 24h peak podeli sa all-time peak-om i na taj način se može sagledati koliko je igra bila uspešna u zadržavanju interesa igrača. Isto, bitno je napomenuti da su ove igre dostupne i na drugim platformama i konzolama (Sve tri su dostupne na Playstation-u i Xbox-u dok je Anno 1800 još dodatno dostupan i na Ubisoft Connect-u) te ove metrike ne pokazuju na celokupnu sliku ali pomoću njih se mogu izvući zaključci i trendovi koji se gotovo sigurno ponavljaju i na tim drugim mestima.

4.4. SWOT Analiza

Analizom tržišnih pokazatelja i metrika prethodno obrađenih naslova nedvosmisleno je utvrđeno da su one bile ogroman uspeh, svaka u svom žanru i reonu. Zbog toga će rad dalje da izvući iz njih uzore, inspiracije i upozorenja uz pomoć kojih bi izveli zaključke koji bi se primenili tokom razvoja igre „Karte i kolonije“. Konkretno će biti izvučene i jasno formatirane snage (strengths), slabosti (weaknesses), šanse/prilike (opportunities) i pretnje (threats) pomenutih igara.

To će biti urađeno putem SWOT analize. Takva analiza je alat koji omogućava sistematsko razmatranje internih i eksternih faktora uspeha nekog proizvoda na tržištu, kako bi se identifikovali elementi koji se mogu usvojiti ili izbeći u sopstvenom dizajnu i strategiji razvoja. U okviru ovoga rada tri igre koje su inspiracije će biti ti proizvodi, a one su: Slay the Spire, Anno 1800 i Kingdom: Two crowns. Svaka od ovih igara je postigla znatan uspeh upotrebom mehanika koje su slične mehanikama igre „Karte i kolonije“. Ovom analizom su se:

- identifikovale korisne mehanike i dobre dizajnerske prakse koje bi mogle da se implementiraju,
- uočile potencijalne greške i slabosti koje treba izbeći,
- prepoznale tržišne niše i prilike za diferencijaciju,
- razumele moguće spoljašnje pretnje koje bi mogle uticati na uspeh igre.

Nakon samog prikaza uporedne SWOT analize biće kraća analiza unutar koje će se izvući ključni uvidi koji će dalje oblikovati razvojni proces i tržišnu strategiju.

4.4.1. SWOT tabela

Naziv igre	Snage (Strengths)	Slabosti (Weaknesses)	Prilike (Opportunities)	Pretnje (Threats)
Slay the Spire	Inovativan deck building sistem Visoki rejting Jasan identitet	Slaba narativna komponenta Minimalistička vizuelna komponenta	Ekspanzije i spin-off igre Potencijal za mobilnu verziju	Velika konkurencija u roguelike žanru Moguće zasićenje publike usled jednostavnosti sistema
Anno 1800	Dubok i kompleksan sistem ekonomije i resursa Vizuelno impresivan Dugačak životni ciklus	Zahtevna za početne igrače Veliki sistemski zahtevi	Ekspanzije Multiplayer mod i podrška	Velika konkurencija u city builder žanru Zavistan od Ubisoft platforme
Kingdom: Two Crowns	Jednostavna kontrola i jaka atmosfera Kooperativni mod kao dodata vrednost Stil privlačan casual igračima	Ograničena kompleksnost Slaba zainteresovanost nakon odigranog jednog ciklusa igre	Potencijal za verzije na drugim konzolama i sistemima Ekspanzije	Naglo opadanje interesa Vizuelna sličnost sa drugim igrama koji koriste pixel-art

4.4.2. Zaključak iz SWOT analize

Na osnovu SWOT analize se mogu izdvojiti i definisati specifične prednosti koje su doprinele uspehu pomenutih igara, ali ujedno i slabosti koje su ograničile ili ometale taj uspeh. Te prednosti i slabosti će poslužiti za izvlačenje brojnih uvida koji će se koristiti prilikom izrade i kasnijeg pozicioniranja igre „Karte i kolonije“ na tržištu.

Od Slay the Spire će se preuzeti čista struktura deck-building mehanike, njena modularnost i lakoća ekspanzije. Njen vizuelno minimalistički stil se međutim na tržištu pokazala kao jedna od najčešćih kritika koja je verovatno uz slabost i nedostatak narativnog elementa odbila ne malu količinu potencijalnih igrača.

Iz Anno 1800 se može zaključiti da je kompleksnost ekonomije mač sa dve oštrice. Sa jedne strane složena ekonomija i gameplay loop koji se vrti oko nje pravi izazove koji veliki broj igrača voli da rešava i prevazilazim povećavajući mogućnost i želju za ponovnim igranjem. Sa druge strane ova složenost drastično otežava novim igračima da nauče igru, što je sigurno uticalo na uspeh i prodaje igre.

Kingdom: two crowns ukazuje na značaj narativnog elementa, postepeno građenje i akumuliranje atmosfere. Jednostavne kontrole i mehanike pružaju lako i brzo učenje igre čineći je privlačnu za igrače koji možda nisu previše iskusni u tom žanru video igara. Glavne zamerke ove igre su da je vizuelni stil nedovoljno jedinstven među igrama koje koriste pixel-art i da nakon što se igra jednom pređe, želja za ponovnim igranjem je doslovno nepostojana.

Svi ovi zaključci se mogu svesti na sledeće prioritete za igru „Karte i kolonije“:

- igra mora imati jasan identitet i atmosferu, estetski i mehanički se mora isticati
- gameplay loop mora biti kompleksan zarad zanimljivosti ali ujedno mora biti dovoljno jednostavan da bi bio lako pristupačan
- deck-building sistem mora biti modularan i fleksibilan kako bi preko njega bile ostvarene gore pomenuta teza kompleksnosti
- sa implementacione strane igra mora biti struktuirana i napravljena tako da omogućiti buduću nadogradnju, bilo zvaničnu u vidu ekspanzija, ili nezvaničnu putem modova, kako bi se produžio životni vek igre i ujedno njena profitabilnost

Ovi uvidi će biti direktno primenjeni u daljim fazama razvoja igre „Karte i kolonije“ i služiće kao smernice kako da se kreira proizvod koji ima ne samo funkcionalni dizajn, već i realan tržišni potencijal u visoko takmičarskom indie okruženju.

5. Izbor tehnologije

Kako bi razvoj video igre bio uspešan i održiv, neophodno je da se pažljivo izaberu odgovarajući alati i tehnologije koje će podržati planirani obim, estetski identitet i funkcionalnosti igre. Zbog toga će se posvetiti ovaj segment procesu pravilnog biranja game engine-a, programskog jezika i dodatnih alata koji će ubrzati razvojni proces, pojednostaviti debug-ovanje i testiranje i kasnije olakšati distribuciju igre na tržištu. Prvo će se prikazati svi mogući game engine-i koji su dostupni, zatim će se napraviti odabir i detaljno obrazložiti taj proces i na kraju će se navesti koje još alati su korišćeni pored game engine-a i zašto.

5.1. Kratko predstavljanje i poređenje dostupnih opcija

Tokom razvoja video igara izbor game engine-a je jedna od najvažnijih odluka koja direktno utiče na tok razvoja, performanse same igre, skalabilnost i dugoročno održavanje projekta. Postoji veliki broj različitih i usko specijalizovanih rešenja, ali se u razvojnom slučaju u okviru ovoga rada tri engine-a ističu kao najbolji kandidati za izbor: Unity engine, Unreal engine i Godot engine. Radi lakšeg donošenja odluke biće napravljena uporedna analiza sa posebnim fokusom na karakteristike relevantne za potrebe razvoja igre „Karte i kolonije“. Te bitne karakteristike su:

- Licenca, njen tip, dužina trajanja i trošak usled nje
- Programski jezici koji se koriste unutar game engine-a
- Prednosti, opšte karakteristike game engine-a koja mu daju prednost nad drugim game engine-ima
- Mane, opšte karakteristike samoga game engine-a koje bi otežale razvoj
- Konkretna upotrebljivost za igru koja se razvija, šta bi se i kako maksimalno upotrebilo tokom razvoja igre, a šta bi bilo ili nepotrebno ili neupotrebljivo

5.1.1. Unity engine

Unity engine ima licencni model zatvorenog (*proprietary*) tipa. U okviru ovakvih tipova licenci cena varira na osnovu određenih unapred ustanovljenih uslova. Na početku razvoja mora da se plati fiksna količina koja zavisi od veličine korisnika. Unity nudi besplatne opcije za manje timove, entuzijaste i studente, dok kod većih kompanija i

timova se plaća na godišnjem nivou. Po završetku proizvoda i njegovog stupanja na tržište plaća se procenat ukupnih profita/prihoda.

Unity je kreiran na osnovu .NET framework-a, što nudi korisnicima par opcija za izbor programskog jezika. Jezik koji dominira u okviru Unity game engine-a je C#. Moguća je upotreba drugih (poput C++) ali tada su ili neke mogućnosti u okviru engine-a onemogućene usled slabe ili nedovoljne podrške ili je neophodno instalirati dodatke za integraciju tih jezika. Radi lakšeg donošenja odluke tokom izbora engine-a koji će se koristiti, a i implementacije, vodiće se da bi bio korišćen isključivo C# u Unity-u.

Jedna od velikih prednosti Unity-a je da ima izuzetno dobro razvijen i bogat ekosistem koji je građen oko njega. Količina tutorijala, kurseva i asset-a koji su građeni za Unity je stvarno enormna, što bi znatno olakšalo učenje okruženja i razvoj unutar njega. Sam engine je izuzetno fleksibilan i stabilan. Može da se modifikuje, doradi i proširi po potrebi. Građen je da podrži razvoj i 2D i 3D igara unutar sebe, a takođe ima i podršku za multiplatformski razvoj, što drastično olakšava prebacivanje igre na druge platforme poput telefona i video konzola.

Uprkos gore navedenim prednostima Unity pak poseduje neke ozbiljne mane. Pre svega nedavna kontroverza oko promene poslovnog modela i tipa licenci. Tokom nje Unity je prvobitno odlučio da promeni tip licence tako da bi se plaćao manji udeo od prihoda ali zato bi se naplaćivalo po svakom skidanju date igre. Ovako promenjenu licencu bi onda i retroaktivno primenili za tekuću fiskalnu godinu nad svim igrama. Ovakav sistem i režim naplate bi bio katastrofalan za određene igrice (pre svega mobilne) kod kojih se primenjuje tip monetizacije kod koga je sama igra besplatna, ali tokom igre bi se ili puštale reklame ili nudile mikrotransakcije, te je proces povraćaja investiranog novca kod ovih igrica spor i postepen. Usled toga mnoge kompanije koje primenjuju samo ovaj stil monetizacije ne bi imale dovoljno kapitala da plate Unity-u i bankrotirale bi. Nakon intenzivnog i jakog suprotstavljanja od klijentele, Unity je pak odlučio da zadrži prvobitan tip monetizacije, ali je poverenje u kompaniju opravdano znatno opalo. Stari sistem licenci, iako bolji, nije savršen. Zbog naplaćivanja procenta prihoda upotreba Unity engine-a predstavlja konstanti trošak koji se nikad ne može pouzdano proračunati. Sa tehničke strane, iako Unity podržava 2D igre, obično zauzima i troši приметно više procesorskih resursa nego što mu i treba, što narušava optimizaciju ovih igara.

U okviru opsega i ciljeva ovoga rada Unity na prvi pogled deluje kao idealan kandidat, međutim poseduje par znatnih prepreka zbog kojih bi bilo pametno da ne požurimo sa donošenjem odluke i da potražimo još kojeg kandidata. Usled nepredvidivog poslovnog sistema praćenog padom poverenja u kompaniju i proizvod došlo je i do postepenog smanjivanja kvaliteta i kvantiteta asset-a i drugih stvari u okviru Unity ekosistema, što je izuzetno loše s obzirom da i Unity najviše zavisi i od njega.

5.1.2. Unreal Engine

Unreal Engine je takođe komercijalni game engine koji poput Unity engine-a koristi model zatvorenog (*proprietary*) tipa, ali ima razlika. Korišćenje engine-a i svih njegovih opcija i alata je potpuno besplatno i na raspolaganju korisnika sve dok projekat koji je napravljen pomoću njega ne ostvari milion dolara prihoda. Od tog momenta izdavač igre je obavezan da plaća 5% daljeg profita. Zbog ovakvog pristupa razvoj manjih i srednjih projekata na početku nije opterećen troškovima, ali zbog toga ako se ostvari znatan komercijalni uspeh traži se primetan deo tih prihoda.

Sa tehničke strane, Unreal Engine je najviše orijentisan ka radu sa C++ jezikom koji predstavlja osnovu većine sistema unutar samoga engine-a. Pored klasičnog razvoja sa programskim jezikom, Unreal Engine nudi i veoma razvijen sistem za vizuelno programiranje, poznat pod nazivom *Blueprints*, koji omogućava direktnu implementaciju video igre bez direktnog pisanja koda. Ovakva kombinacija sistema daje veliku fleksibilnost ali ujedno i povećava kompleksnost okruženja i samim tim engine postaje zahtevniji za baratanje i razumevanje.

Ubedljivo najveće prednosti Unreal Engine-a su vezane za grafiku i napredne sisteme za vizuelizaciju. Unreal važi kao jedan od najmoćnijih alata za rendering na tržištu i poseduje izvanredne sisteme za osvetljenje, fiziku, animacije i vizuelne efekte. Kao i Unity, Unreal ima bogat ekosistem dodataka i dobru integraciju sa drugim alatima i tehnologijama koje se koriste u industriji (DCC alati za 3D modele i animacije, razni sistemi za verzionisanje koda i mnoge druge).

Sve ove pozitivne karakteristike koje čine Unreal fenomenalnim izborom za 3D projekte ujedno predstavljaju i njegove mane kada su u pitanju 2D projekti manjeg obima. Usled njegove složenosti i velikog broja podsistema, Unreal je jako zahtevan kada su u pitanju računarski resursi. Takođe ova složenost otežava prvobitno savladavanje svih

moгуćnosti i opcija, što znatno usporava prvobitne faze projekta, ali zato olakšava i ubrzava kasniji razvoj. Na žalost, kod manjih projekata ova prednost je zapravo mana pošto usled jednostavnosti ovih projekata veći deo produkcije je u toj prvobitnoj fazi, što znači da Unreal Engine u stvari usporava razvoj manjih igara bez neke preterane dobiti.

Zarad postizanja ciljeva ovoga rada Unreal Engine se može posmatrati kao previše moćna alatka koja je ujedno i previše zahtevna u odnosu na same potrebe projekta. Iako je razvoj igre iz ovoga rada moguć, najbolje i najjače karakteristike Unreal-a ne bi bile uopšte upotrebljene. Na osnovu svega ovoga, Unreal se može sumirati kao fantastičan izbor za 3D projekte srednjeg ili većeg obima, ali kao ne baš najbolji izbor za 2D igru malih razmera i kompleksnosti.

5.1.3. Godot engine

Godot je open-source game engine koji koristi MIT licencu, što znači da je korišćenje engine-a potpuno besplatno i da je izvorni kod javno dostupan za pregled, izmenu i prilagođavanje. Ovakav tip licenciranja eliminiše neizvesnost oko troškova i omogućava timovima da dugoročno planiraju razvoj bez dodatnog finansijskog pritiska vezanog za samu tehnologiju.

Godot podržava više programskih jezika: C#, C++ i Rust, ali je njegov primarni jezik GDScript. To je skriptni jezik sličan i delimično zasnovan na Python-u. GDScript je jednostavan za učenje, čitljiv i usko povezan sa editorom i sistemom čvorova (*nodes*), što ga čini pogodnim i za brzu izradu prototipova i za vođenje većih projekata. Mogućnost kombinovanja GDScript-a sa drugim jezicima kao što su C# ili C++ dodatno povećava fleksibilnost tokom razvoja.

Glavna prednost Godot-a jeste to što je lagan, brz i posebno optimizovan za 2D projekte. Editor je intuitivan, korisnički interfejs je pregledan a sistem scena i čvorova omogućava visok stepen modularnosti što omogućava ponovnu upotrebu koda. Usled otvorene arhitekture i dostupnosti izvornog koda korisnik ima veliku kontrolu nad ponašanjem engine-a i implementacijom sopstvenih podsistema. Zbog toga se Godot često ističe kao idealna platforma za eksperimentisanje, istraživanje različitih pristupa dizajnu i za edukaciju u oblasti razvoja igara.

Mimo svega ovoga Godot ima i određena ograničenja. U poređenju sa Unity-em i Unreal-om, zajednica korisnika je manja, pa je i broj dostupnih tutorijala, komercijalnih

kurseva i gotovih asset-a skromniji. Podrška za naprednije 3D grafičke sisteme još uvek je u ranoj fazi razvoja i trenutno ne dostiže nivo naprednosti najnovijih verzija Unity-a i Unreal-a. Dodatno, Godot je mlađi engine, pa su i njegova tržišna prisutnost i poverenje u okviru industrije u nešto skromnijoj meri razvijeni.

U kontekstu igre koju razvijamo, Godot se pokazuje kao veoma dobar kandidat. Reč je o 2D igri sa fokusom na modularne sisteme, kao što su deck-building mehanike, proceduralna generacija sadržaja i bogata interakcija kartica i drugih entiteta u igri. Za ovakav tip projekta posebno su značajne fleksibilnost, jasnoća i jednostavnost rada sa 2D okruženjem i mogućnost da se lako uvode i menjaju novi podsistemi. Godot nudi dobar odnos između jednostavnosti i moći bez nametanja dodatnih troškova licenciranja.

Na osnovu uporedne analize Unity-a, Unreal-a i Godot-a, kao i specifičnih potreba projekta, u narednom segmentu biće doneta i obrazložena konačna odluka o izboru game engine-a koji će biti korišćen kao glavno razvojno okruženje.

5.2. Izbor in-game engine-a i programskog jezika

Uz pomoć prethodne detaljne analize game engine-a koji su na raspolaganju, opredelilo se da igra „Karte i kolonije“ bude razvijena u Godot engine-u koristeći GDScript kao glavni programerski jezik. Ova odluka je bila doneta u skladu sa sledećim tehničkim, dizajnerskim, praktičnim i organizacionim zahtevima projekta:

- *Igra je 2D* – Godot nudi ubedljivo najbolje okruženje za razvoj 2D igara koje je izuzetno optimizovano i lako za upotrebu. Godot nudi lakši debugging i veću kontrolu nad dizajnom i mehanikama 2D igara u odnosu na ostale, drastično ubrzavajući pravljenje iteracija igre „Karte i kolonije“. Sve ovo bi imalo znatan pozitivan uticaj na proces razvoja same igre.
- *Modularna struktura sistema* – s obzirom da su glavne mehanike u okviru igre (sistem za deck building, proceduralna generacija događaja, sistem resursa) višeslojne i trebaju međusobno da interaguju, bilo bi poželjno da se ima okruženje koje nudi hijerarhijski i modularan sistem za razvoj. Godot upravo nudi ovakav sistem koji je takođe pregledan i skriptabilan, čineći ga još privlačnijim.

- *Brza kriva učenja* – budući da se koristi interni jezik GDScript koji podseća na Python čini razvojni proces pristupačnijim i bržim, pogotovo za implementaciju gameplay mehanika, testiranja i izradu prototipa.
- *Open source i fleksibilnost* – Godot je kompletno open source projekat, što daje mogućnost modifikacije engine-a u slučaju ako se javi potreba, a da ne mora da se plaća licenca ili neka druga vrstu naknade. Ovo čini Godot pogodnim za indie i akademske projekte.

Što se tiče izbora jezika, GDScript je odabran jer:

- je sintaksički *jednostavan, čitljiv, pregledan i intuitivan*
- integrisan je direktno u Godot, što omogućava najbolju kompatibilnost
- podržava objektno orijentisani pristup, kao i funkcionalne koncepte koji olakšavaju upravljanje sa složenijim sistemima igre

Iako Godot podržava i rad sa drugim jezicima (poput C# i C++) u okviru potreba ovog rada zarad ispunjenja ciljeva GDScript je i više nego dovoljan za implementaciju potreba, lak je i direktno povezan sa samim editorom zbog čega je ubedljivo najbolji izbor.

5.3. Alati korišćeni tokom razvoja

Za razvoj igre „Karte i kolonije“ koristiće se omanji skup alata koji bi olakšao implementaciju i obezbedio osnovne i neophodne potrebe poput verzionisanja koda, rad u samome engine-u, dizajn korisničkog interfejsa, pisanje dokumentacije i distribuciju razvojnih verzija.

Za verzionisanje koda koristiće se *Git* i *GitHub*. *GitHub* je centralna baza podataka gde se skladišti sav kod koji je pisan. U okviru njega, glavna grana (*main*) će čuvati sve stabilne verzije igre, dok će pojedine funkcionalnosti biti razvijane u odvojenim granama koje bi kasnije bile spojene sa glavnom. Prilikom spajanja pišaće se kratke, jasne i konkretne *commit* poruke, kako bi se kasnije moglo jasno videti šta je odrađeno u kojoj iteraciji i da bi kasniji pregled razvoja i koda bio lakši za izvođenje.

Glavno razvojno okruženje će biti Godot engine sa fokusom na razvoj i *export-ovanje* na Windows desktop. Godot već u sebi ima ugrađene alate: *Profiler* i *Debugger* koji će se koristiti za praćenje performansi i toka izvršavanja igre. Takođe će se koristiti *GDScript linter* i *formater* koji će poslužiti da stil koda ostane dosledan tokom razvoja.

Za razvoj UI i UX elemenata takođe će se koristiti Godot editor s obzirom da ima dobre alate upravo specijalizovane za takav razvoj. UI prototipovi će biti razvijeni uz pomoć *Control* čvorova koji omogućavaju da se vidi kako će ekran tokom igre izgledati i kako će se ponašati bez prethodne kompilacije. Tako će se smanjiti vreme potrebno za razvoj prototipova i moći će brže, bolje i lakše da se uoči i koriguje bilo kakva greška.

Dokumentacija projekta je napisana u okviru ovog Word dokumenta. Po potrebi će biti ubačen po koji dijagram ili tabela zarad lakšeg razumevanja i vizuelizacije određenih ideja, sistema i pojmova.

Što se tiče distribucije ove igre koristiće se *export* funkcija koja je ugrađena u okviru Godot engine-a u vidu razvojnih ili ti *dev build*-ova za Windows i takve build-ovi bi bili prosleđeni zainteresovanim testerima.

6. Tehnička implementacija

Cilj ovog poglavlja je da prikaže proces prevođenja dosadašnjih ideja, planova i koncepata u konkretnu, pouzdanu i modularnu implementaciju pomoću Godot game engine-a i njegovog okruženja. Ovo će se postići detaljnim prikazom postupka realizacije koji bi se izvodio uz pomoć arhitekture scena i čvorova, organizacije koda i fajlova, data-driven modela i disciplinovanog procesa testiranja.

Prilikom izrade projekta vodiće se računa da se prate sledeći principi: *modularnost* (svaki podsistem ima jasnu odgovornost), *data-driven pristup* (balans i sadržaj iz podatka, a ne iz koda), *determinističnost* (seed-ovan pomoću generatora slučajnih brojeva - RNG za reprodukovanje stanja), *testabilnost* (mogućnost izolovanog pokretanja i verifikacije sistema) i *performanse* (2D-orijentisana optimizacija i izbegavanje nepotrebnih alokacija). U skladu s tim, poglavlje je strukturisano na sledeći način:

- 6.1 Arhitektura projekta – opšti dijagram podсистema i njihovih granica. Upravljanje sesijom (game flow/state), DeckManager, ResourceManager, EventManager ..., sloj za UI i HUD i veze putem signala. Ukratko će biti opisani specifični Godot obrasci (scene i čvorove, autoload i singletoni) i dogovor o ciklusu ažuriranja (tick/step).
- 6.2 Organizacija koda i fajlova – hijerarhija direktorijuma (poput scenes/, scripts/, data/, ui/, assets/, tests/), konvencije imenovanja (klase, resursi, prefiksi), pravila za grananje/faziranje funkcionalnosti i kako će se održavati razdvojenost UI-a od logike igre.
- 6.3 Data driven sistem – model podataka za karte, događaje i resurse (format polja, tagovi, retkost, trošak, efekti), način učitavanja (npr. .tres/.res ili JSON), validacija i balans bez rekompajliranja. Biće objašnjeno kako se parametri (npr. verovatnoće događaja) podešavaju iz fajlova i kako seed-ovan RNG obezbeđuje reprodukciju.

Po završetku ovog poglavlja, čitalac treba jasno da vidi: kako se sistem deli na celine, gde se nalazi koji deo igre, kako se sadržaj menja bez diranja koda i kako se funkcionalnost proverava i ponavlja bez narušavanja stabilnosti projekta.

6.1. Arhitektura projekta

U okviru ovog segmenta biće prikazana arhitektura sistema koji je bio kreiran, tj. kako je podeljen na podsisteme, kako ti podsistemi međusobno komuniciraju i takođe će se prikazati obrasci u okviru Godot okruženja koji i omogućavaju ovu arhitekturu (pre svega scene i čvorove, autoload skripte koje su suštinski singleton-i i signale koji služe za komunikaciju između podsistema). Primarni cilj rada je da se pridržava prethodno navedenih ciljeva da igra „Karte i kolonije“ bude: modularna, deterministična, laka za testiranje i da ima minimalno preplitanje između UI segmenata i domenske logike zarad lakšeg praćenja, budućih proširenja i modifikacija.

6.1.1. Pregled podsistema, njihovih odgovornosti, međusobne komunikacije i načina implementacije

Podsistemi igre koji su razvijeni su organizovani u i implementirani pomoću GDScript *autoload* skripti, posebne podvrste skripti u okviru Godot engine-a koji se učitavaju automatski (otuda i naziv) pri pokretanju igre. One se suštinski ponašaju kao singleton-i, klase koje su instancirane samo jednom tokom rada celog programa. Zbog ovoga, kao i činjenice da se može relativno lako i globalno pristupiti podacima u okviru njih, se omogućava pouzdano i čitko čuvanje podataka na jednom mestu, što dalje omogućava održavanje, modifikovanje i praćenje stanja igre koje je onda izraženo u tim podacima. Odlučeno je da se naprave menadžeri ključnih mehanika igre koji bi bili upravo ove autoload skripte. U njima bi se čuvali podaci koji bi bili relevantni za datu mehaniku i koji bi predstavljali dato stanje igre u tom trenutku (na primer u okviru menadžera za karte čuvale bi se koje sve karte se drže u ruci, a koje su preostale u špilju). Ovi podaci bi zatim bili prosleđeni pomoću signala do UI sloja koji bi ih dalje prikazao korisniku. Sada će detaljnije biti opisane i predstavljene date skripte.

Game skripta je zadužena za čuvanje globalnih podataka o trenutnoj sesiji igre. U okviru nje se nalaze informacije o tome na kom je potezu igrač, koliko ima raspoloživih resursa, koji su trenutno aktivni efekti i takozvani *RNG seed* koji je bio korišćen za generisanje nasumičnih vrednosti u datoj sesiji. Koristi dva signala za komunikaciju sa ostatkom igre: *turn_changed* i *resources_changed*. Njih UI i ostali menadžeri mogu da

osluškiju kako bi mogli da osveže prikaz ili da pokrenu dodatnu logiku kada se u igri završi potez odnosno kada dođe do promene količine resursa.

DeckManager je skripta koja se bavi upravljanjem špilom. U njoj se čuvaju sve trenutno aktivne karte, bez obzira na to da li je igra u talonu, ruci ili u deponiji. Pored samih podataka o kartama, u ovom podsistemu se nalaze i pravila za vučenje i igranje karata, a isto ima i sistem za proveru da li može da se odigra neka karta u datom trenutku. Ova skripta šalje jedan signal: *hand_changed* koji UI sluša kako bi mogao da prikaže promene koje nastaju kada se neka karta odigra ili kada se vuku nove karte.

BuildingManager je zadužena za zgrade. Ona prati koje su zgrade trenutno izgrađene, do kog nivoa su unapređene i koje efekte pružaju. U direktnoj je komunikaciji sa *DeckManager*-om s obzirom da su zgrade zapravo efekti određenih karata i kada dođe do izgradnje ili unapređenja određene zgrade dolazi i do unapređenja određenih karata koje su relevantne za taj tip zgrade. Šalje signal *building_changed* kada dođe do bilo kakve promene, a taj signal osluškuje UI i drugi podsistemi kako bi mogle da adekvatno prikažu tu promenu ili da dodatno aktiviraju neki podsistem.

EventManager se bavi praćenjem stanja događaja u datoj igri. U okviru nje, učitani događaji se prvo grupišu po nivoima (odnosno *tier*-ovima) u zavisnosti od njihove složenosti, uslova za aktivaciju i efekata. Na početku svakog poteza prvobitno gleda šansu da se desi događaj i, u slučaju ako može, proverava koji događaji ispunjavaju uslove koje im omogućavaju da budu odigrani a zatim nasumično bira jedan iz skupa omogućenih. Događaji utiču na više sistema odjednom ili zahtevaju uslove iz tih drugih sistema, pa zato *EventManager* ima veliki broj signala za komunikaciju, a najbitniji su: *event_started*, *event_ended*, *active_events*, *active_events_changed*, *choice_required* i *objective_updated* a njih osluškuju UI i svi ostali sistemi kako bi mogli ili da predstave promene ili da ih proprate sa svojim.

Odvojeno od svih ovih podsistema nalazi se sloj za UI koji je implementiram pomoću UI *čvorova* i *scena*. UI prikazuje sve prethodne podatke koji se nalaze u menadžerima ali nije zadužena za skladištenje. Bitno je naglasiti i način kako ti podaci dospevaju do UI-a. Umesto da se u okviru raznih podsistema pozivaju UI elementi ili atributi odlučeno je da se šalju signali koji UI osluškuje, prihvata podatke kada se oglase, raspakuje ih i onda upisuje primljene podatke iz signala na odgovarajuća mesta. Ovako se skroz razdvojio

UI sloj od menadžera/implementacionog sloja što povećava fleksibilnost projekta i drastično olakšava refaktorisanje koda sa bilo koje strane. Ovaj stil komunikacije se zove *event-driven* model.

6.1.2. Tok stanja igre

Tok igre od početka jedne sesije do samoga kraja bi mogao da se organizuje u niz jasno definisanih i jednostavnih poteza.

Nakon pokretanja same igre, kreira se nova sesija pozivom funkcije *NewSession(seed)*. Tada se inicijalizuju globalne promenljive i opšti podaci o partiji, postavlja se početni *RNG seed* i pripremaju se početni resursi, špil i početno stanje zgrada i događaja.

Sledi igranje poteza. Svaki potez počinje pozivanjem funkcije *StartTurn*, pri čemu se broj poteza uvećava i emituje se signal *turns_changed*. Korisnički interfejs koji osluškuje dati signal zatim osvežava prikaz poteza. Sledi funkcija *RefillHand* u kojoj *DeckManager* popunjava ruku igrača do maksimalne količine karata. Kada se ruka napuni šalje se signal *hand_changed*, koji UI osluškuje i reaguje ažuriranjem prikaza karata na ekranu.

Nakon što je ruka spremna, igrač ulazi u deo igre *PlayerActions*, tokom koje može da odigra svoje karte koje ima na raspolaganju u okviru svog jednog poteza. Svaka igranje karte može da ima efekte koji utiču na resurse, zgrade ili aktivne događaje. Kada dođe do promene u raspoloživim resursima, *Game* šalje signal po imenu *resources_changed* kako bi UI i drugi podsistemi koji osluškiju dati signal mogli da se prilagode novoj situaciji. U slučaju da određeni događaji zahtevaju izbor igrača (u okviru rada su to samo *tier 2* događaji sa više mogućih ishoda), *EventManager* emituje signal *choice_required* na šta UI reaguje prikazom odgovarajućeg ekrana sa izborima.

Kada igrač završi sa igranjem karata, poziva se *EndTurn*, u okviru kog se izvršava faza održavanja (ili ti *upkeep*). U njoj se obračunava potrošnja i produkcija resursa, primenjuju se vremenski ograničeni efekti i pokreću se takozvane *tick* logike pojedinih događaja i zgrada (da se nešto jednom po potezu ili ti otkućaju desi).

Kao što se može videti igra je koncipirana tako da tok igre ostaje jasan i linearan, dok komunikacija između podсистema ide preko signala. To omogućava da se logika toka igre razvija i testira nezavisno od konkretne implementacije korisničkog interfejsa, koji samo prati emitovane događaje i prilagođava prikaz.

6.1.3. Godot obrasci: scene, čvorovi i teme

Igra ima tipičan Godot UI dizajn sa jednom početnom scenom iz koje se ostale scene i elementi dalje instanciraju. U igri „Karte i kolonije“ je to *Main* scena. Ona je čvor tipa *Node2D* i služi kao koren čitavog UI-a. U okviru nje se nalaze reference na menadžere i glavni korisnički interfejs. Menadžerski deo je predstavljen kroz čvor *Deck* (direktna veza sa skriptom), dok je UI grupisan u posebnom čvoru tipa *CanvasLayer*, kako bi se interfejs uvek bio nezavistan.

Unutar UI čvora organizacija je sledeća:

Na vrhu se nalazi *SafeArea* koja osigurava da se UI prikazuje na ekranima na isti način, bez obzira na njihovu rezoluciju.

Unutar njega nalazi se *RootVBox* kao glavni vertikalni raspored. U gornjem delu *RootVBox*-a nalaze se *ResourceBar* i *EventBar*, koji prikazuju resurse i aktivne događaje.

Centralni deo zauzima *CenterMap*, odnosno zona za prikaz same „mape“ ili prostora u kome se odvija igra (koja je za sada prazna).

Sa leve i desne strane nalaze se paneli *BuildingsPanel* i *ObjectivesPanel* koji prikazuju stanje zgrada i trenutne ciljeve.

U donjem delu ekrana je *HandBar* koji je zadužen za prikaz karata koje su u ruci.

Kako bi se uzeo i obeležio izbor igrača tokom *tier 2* event-ova koristi se modalni prozor *EventChoiceModal* koji se po potrebi instancira kao dete UI sloja i na taj način privremeno preuzima fokus interakcije.

Koristi se centralna tema za korisnički interfejs kako bi izgled bio isti i konstantan. To je globalni resurs *MainTheme.tres* koji se postavlja kao glavna tema za UI. Na osnovu njega se prave *type variations* za različite delove interfejsa čime postiže da svi elementi koji pripadaju istom „tipu“ dele zajednički stil (iste boje, fontove i okvire). Ovim je izbegnuto ručno podešavanje stila na nivou pojedinačnih čvorova što bi to otežalo kasnije održavanje i menjanje. Većina vizuelnih promena donosi se kroz centralnu temu tako da se promena stila može uraditi na jednom mestu i automatski će ostali elementi da se update-uju.

Ovakav pristup scenama, čvorovima i temama uklapa se u ciljeve projekta. UI hijerarhija je jasna i laka za snalaženje, stil je konzistentan a promene u izgledu ne zahtevaju prolazak kroz desetine pojedinačnih čvorova i ručno preklapanje podešavanja.

6.1.4. Determinizam i mogućnost ponavljanja

Da bi igra bila lakša za testiranje napravljena tako da se ključne nasumične odluke zasnivaju na jednom centralnom generatoru slučajnih brojeva (*RNG*). Kada se pokrene sesija igre kreira se jedinstveni *seed* koji se koristi kao početna vrednost za *RNG*. Taj *seed* može ručno da se postavi, a ako nije postavljen igra ga generiše automatski. On se zatim prosleđuje podsistemima koji koriste slučajnost u svojim funkcijama a oni su: *DeckManager*-u (za vučenje karata) i *EventManager*-u (za izbor događaja).

Uz pomoć ovakvog pristupa sesija postaje determinisana i kontrolišemo nasumični aspekt igre. Ukoliko je igra pokrenuta sa istim *seed*-om i igrač ponovi iste poteze, tok partije i svi nasumični izbori biće isti.

6.1.5. Performanse

Vodiće se računa o poboljšavanju i održavanju zadovoljavajućih performansi tako što se izbegava nepotrebno opterećenje sa 2D prikazima i korisničkim interfejsima. U UI skriptama se ne koriste *_process()* funkcije kao opšti mehanizam za osvežavanje nego interfejs koristi isključivo signale. Kada se promeni neko stanje u igri odgovarajući menadžer emituje signal a UI u tom trenutku osvežava prikaz. Ovako se izbegava stalna provera stanja u svakom frame-u i drastično se smanjuje konstantan rad, a samim tim i opterećenje igre.

U delovima sistema koji se često izvršavaju vodi se računa o izbegavanju suvišnih alokacija objekata. Gde god je razumno recikliraju se strukture podataka umesto da se stalno kreiraju nove instance, čime se smanjuje pritisak na *garbage collector*-a i izbegavaju se nepotrebne pauze u izvršavanju. Pored toga, *layout* korisničkog interfejsa dizajniran je da bude stabilan. Koriste se procentualni raspored umesto fiksni veličina, a margine i razmaci su definisani tako da se ne pomeraju pri *hover* ili *pressed* stanjima dugmića. Ovo doprinosi i vizuelnoj konzistentnosti i smanjenju ponovnog iscrtavanja elemenata.

Za praćenje performansi koristi se ugrađeni *Profiler* u Godot-u. Znatna pažnja je bila udeljena da bi čvorovi interfejsa što duže trajali tako što se recikliraju (odnosno ubacuju se novi podaci u postojeće elemente a ne prave se nove instance čvora) kao i ponašanju *garbage collector*-a tokom intenzivnih sekvenci (na primer kada igrač u kratkom

vremenskom periodu odigra veliki broj karata). Ovakav pristup omogućava da se eventualni problemi u performansama uoče rano i da se ciljano optimizuju kritični delovi sistema, bez nepotrebne optimizacije ostatka koda.

6.1.6. Mogućnost nadograđivanja i data-driven pristup

Glavni cilj arhitekture je olakša bilo koje proširivanje novim sadržajem bez potrebe za izmenama koda. Zbog toga su karte i događaji implementirani kao posebni resursi koji se učitavaju iz fajlova. Podsystemi *CardDB* i *EventDB* su zaduženi da pri pokretanju igre pročitaju te resurse, da izvrše proveru da vide da li su podaci ispravni i da ih proslede odgovarajućim menadžerima. Sa ovakvim pristupom dodavanje nove karte ili događaja se svodi na kreiranje još jednog *resource* fajla sa odgovarajućim parametrima, bez potrebe da se u kodu dodaje išta.

Ova logika takođe važi i za zgrade i njihove efekte. Umesto da efekti unapređivanja zgrada budu hardcode-ovani direktno u kodu oni su opisani parametrima koji se nalaze u definicijama efekata. Na primer, promena špila može da se zabeleži u okviru jedne karte preko parametara tipa *deck_add* ili *deck_remove*, kojima se zadaju koje karte treba dodati ili ukloniti kada zgrada pređe na novi nivo. Na ovaj način, ako se kasnije promeni balans igre ili se uvede novi nivo zgrade, dovoljno je izmeniti odgovarajuće vrednosti u podacima a da se ne dira kod menadžera.

Sloj interfejsa je takođe organizovan tako da podržava širenje sadržaja. UI paneli ne komuniciraju direktno jedni s drugima a ni sa internom strukturom menadžera. Oni zapravo reaguju na signale koje menadžeri emituju. To znači da dodavanje novog panela (na primer dodatnog panela za statistiku ili istoriju poteza) uglavnom podrazumeva kreiranje nove scene, pravljenjem metoda kako će da osluškuje već postojeće signale i implementaciju načina na koji će panel prikazivati podatke koje primi od osluškivanja. Postojeći menadžeri zbog ovoga ne moraju da se menjaju. Ovo pojednostavljuje proširenja i održavanje, jer se novi sadržaj uvodi bez rizika da se naruši postojeće ponašanje sistema.

6.2. Organizacija koda i fajlova

Dobra organizacija samoga koda i projektnih fajlova u kojima je čuvan je neophodno kako bi neki projekat bio pregledan i samim tim lak za rad. Znatno olakšava dodavanje novog sadržaja. Kako bi adekvatno bio organizovan ovaj projekat neophodno je jasno definisati strukturu direktorijuma, postarati se da postoji minimalno preplitanje UI sloja i domenske logike i da se podrži data-driven pristup kojim se sadržaj menja kroz podatke, a ne kroz kod.

U narednim potpoglavljima biće prikazana hijerarhija direktorijuma (na primer *scenes/*, *scripts/*, *data/*, *ui/*, *assets/*, *tests/*), osnovne konvencije imenovanja i pravila koja su korišćena pri razdvajanju korisničkog interfejsa od logike igre. Na ovaj način čitalac može lako da stekne sliku o tome gde se šta nalazi u projektu i kako se projekat održava i proširuje tokom razvoja.

6.2.1. Struktura direktorijuma

Struktura direktorijuma je podešena tako da jasno razdvoji podatke, scene, kod i korisnički interfejs. Na vrhu projekta nalaze se direktorijumi: *assets/*, *data/*, *scenes/*, *scripts/* i *UI/*, od kojih svaki ima jasnu ulogu u sistemu.

Direktorijum *assets/* sadrži sve multimedijalne resurse koji se koriste u igri. U okviru njega folder *fonts/* čuva korišćene fontove za stil. Na ovaj način svi resursi koji nisu direktno vezani za logiku igre, već za njen vizuelni izgled, grupisani su na jednom mestu.

Direktorijum *data/* je zadužen za data-driven sadržaj. U njemu se čuvaju definicije karata i događaja. U poddirektorijumu *data/cards/* nalaze se resursi koji služe za različite tipove karata. Folder *starter_cards/* sadrži sve karte koje ulaze u početni špil na nultom potezu, dok *wood_building_chain/* obuhvata karte vezane za lanac zgrada za drvo, bilo da je reč o samim zgradama ili unapređenim kartama resursa koje proizilaze iz tog lanca. Isto tako u *data/events/* nalaze se definicije događaja podeljene po složenosti: *T1Events/*, *T2Events/* i *T3Events/* čuvaju događaje prvog, drugog i trećeg stepena složenosti. Sve ove definicije čuvaju se kao resursi (.tres ili .res), koje pri pokretanju igre učitavaju CardDB i EventDB.

Sve scene projekta smeštene su u direktorijum *scenes/*. Glavna scena igre je *Main.tscn*, koja je povezana sa odgovarajućom *Main* skriptom i predstavlja ulaznu tačku

za prikaz trenutnog stanja igre. UI scene bi mogle dodatno da budu organizovane u *poddirektorijum-e* (npr. *scenes/ui/*), ali osnovno pravilo je da se sve Godot scene (ekrani, paneli, modali) nalaze upravo u ovom direktorijumu, a ne pomešane sa kodom ili podacima.

Direktorijum *scripts/* okuplja sav kod i podeljen je po slojevima. U *scripts/data/* nalaze se skripte koje definišu prilagođene resurse odnosno custom resources-e. U okviru ove igre na primer *scripts/data/cards/Card.gd* opisuje strukturu podatka koji predstavlja kartu, dok se u *scripts/data/events/* nalaze skripte za definisanje resursa događaja i njihovih podsegmenata (npr. izbora kod tier 2 događaja). U poddirektorijumu *scripts/singletons/* smeštene su pomoćne skripte za mehanike i njihove menadžere koji se učitavaju kao *autoload* i u kojima se čuvaju odgovarajući podaci igre (na primer *CardDB*, *EventDB*, *Game*, *BuildingManager...*). Direktorijum *scripts/systems/* čuva skripte koje implementiraju glavne sisteme igre, kao što su *DeckManager.gd* u *scripts/systems/deck/* i *EventManager.gd* u *scripts/systems/events/*. Na samome kraju, *scripts/UI/* sadrži skripte dodeljene UI scenama i pojedinačnim elementima interfejsa. One služe da povežu prikaz sa podacima iz sistema, ali bez preuzimanja domenske logike.

Vizuelni sloj je dodatno organizovan u direktorijumu *UI/*, gde se nalaze scene i teme korisničkog interfejsa. U *UI/theme/* čuvaju se teme za UI elemente poput centralnog resursa *MainTheme.tres* i njegove varijacije po tipu (na primer za gornju traku, bočne panele, traku sa kartama i slično). Pored tema, u *UI/* se nalaze i različiti *.tscn* fajlovi koji predstavljaju UI scene i u kojima je definisan konkretan raspored elemenata na ekranu.

Prateći ova pravila, projekat ostaje konzistentan:

- sve scene se nalaze u *scenes/* (UI scene u odgovarajućim poddirektorijumima),
- sav kod je u *scripts/* (podeljen na podsisteme i UI),
- svi podaci o kartama, događajima i konfiguracijama nalaze se u *data/*,
- teme i stilovi UI-ja grupisani su u *UI/theme/*

Ovakva struktura olakšava snalaženje u projektu i dodavanje novog sadržaja.

6.2.2. Pravila i konvencije korišćene prilikom imenovanja

Radi lakše implementacije koda, urednosti projekta i olakšanja pri pretrazi projekta tokom samoga razvoja prisvojen je i poštovan skup jednostavnih i ustanovljenih konvencija imenovanja. Scene u Godot-u se imenuju u *PascalCase* formatu, pri čemu naziv fajla objašnjavaju ulogu scene (na primer *BuildingsPanel.tscn*, *EventChoiceModal.tscn*). Na sličan način, skripte koje sadrže glavnu klasu takođe koriste *PascalCase* (*DeckManager.gd*, *EventManager.gd*), a naziv fajla odgovara nazivu glavne klase u skripti. Ovo olakšava pretragu jer je odmah jasno koja skripta je vezana za koju scenu ili podsistem.

Podaci i resursi koji predstavljaju sadržaj igre (karte, događaji i konfiguracije) koriste *lower_snake_case* format. Na primer resurs kartice može da se zove *build_logging_camp.tres*, dok se identifikatori unutar tih podataka (kao što su *building_id* ili *effect_type*) takođe zapisuju malim slovima sa donjom crtom, na primer *building_id* = "wood_logging" ili *effect_type* = "building". Ovakav pristup smanjuje mogućnost da dođe do greške prilikom unosa i olakšava razlikovanje između kodiranih elemenata (klasa, scena) i podataka.

Za teme i stilove korisničkog interfejsa odlučeno je da *StyleBox* resursi koriste prefiks *SB_* u kombinaciji sa deskriptivnim nazivom (na primer *SB_TopBar*), uz konzistentno korišćenje istih *content_margin_** vrednosti za elemente istog tipa. Na taj način je jasno koji resurs je zadužen za koji deo UI-ja, a vizuelni stil ostaje ujednačen kroz ceo interfejs.

Nazivi signala prate obrazac glagola ili radnje u proširenom obliku, što olakšava razumevanje kada se koji signal emituje. Na primer, *hand_changed*, *resources_changed* i *choice_required* direktno opisuju šta se dogodilo u sistemu: da je ruka promenjena, da su resursi ažurirani ili da je potreban izbor igrača. Ovakvo imenovanje dodatno pomaže pri čitanju koda i povezivanju signala sa UI elementima koji na njih reaguju.

6.2.3. Data-driven sistem (učitavanje i validacija)

Kao što je već pomenutoi karte i događaji u igri su modelovani kao podaci odnosno kao posebni resursi koji se učitavaju pri pokretanju projekta. U praksi su korišćeni Godot resurs fajlovi (.tres) povezani sa odgovarajućim *script_class* definicijama. Na primer skripta *Card.gd* deklariše attribute resursa *Card* što znači da svaki fajl koji ima konkretne vrednosti za date attribute je instanca klase *Card* odnosno svaki taj fajl je zapravo po jedna nezavisna karta. Na sličan način definišu se i resursi za događaje, tako da svaki .tres fajl sadrži sve potrebne parametre za konkretnu kartu ili događaj.

Učitavanje ovih resursa obavlja se centralizovano kroz *CardDB* i *EventDB*. *CardDB* pri inicijalizaciji rekurzivno prolazi kroz direktorijum *data/cards/*, učitava sve pronađene resurse i proverava da li svaki od njih ima validan identifikator (ID). Kada naiđe na ispravnu kartu, smešta je u internu mapu *cards_by_id*, gde je ključ ID a vrednost je instanca resursa. Na taj način ostatak sistema može uvek da se osloni na jedinstveni skup karata kojima pristupa preko njihovih identifikatora, bez direktnog rada sa fajl sistemom. *EventDB* funkcioniše na isti način, ali za resurse u *data/events/*.

Pri učitavanju podataka primenjuje se jednostavan, ali koristan sloj za validaciju. Ukoliko resurs nema ID sistem ispisuje upozorenje u logu i preskače takav fajl kako se ne bi pojavio objekat bez jedinstvene reference za sebe u igri. U slučaju da se pojavi dupliran id takođe se beleži upozorenje a zadržava se prva učitana definicija. Ovako se jasno vidi da postoji konflikt u podacima, ali se projekat i dalje može pokrenuti u razvojnoj fazi. Za određene tipove efekata, kao što su oni vezani za zgrade (npr. efekat tipa "building"), očekuju se dodatni obavezni ključevi, poput *building_id* i *to_level*. Ako oni nedostaju, opet se ispisuje upozorenje kako bi se na vreme uočile greške u konfiguraciji.

Prednost ovakvog pristupa je što balans igre može da se menja bez ponovnog rekompajliranja koda. Vrednosti troškova, efekata ili uslova za aktivaciju događaja čuvaju se u .tres fajlovima, pa se mogu podešavati direktno iz editora ili ručnom izmenom tih fajlova. Pri sledećem pokretanju igre engine će ponovo učitati sve resurse i primeniti nove vrednosti. To omogućava brze iteracije na dizajnu i balansu, dok kod ostaje relativno stabilan i fokusiran na strukturu sistema, a ne na konkretne brojke.

Primer – prikupljanje .tres file-ova

```
func collect_card_paths(dir_path: String, recursive: bool) -> Array[String]:
    var all_paths_to_cards: Array[String] = []
    var directory := DirAccess.open(dir_path)
    if directory == null:
        push_warning("CardDB ne moze da otvori folder: %s" % dir_path)
        return all_paths_to_cards

    directory.list_dir_begin()
    while true:
        var card_name := directory.get_next()

        if card_name == "":
            break # kraj iteracije/nema sta vise da ucita

        if card_name.begins_with("."):
            continue # preskačemo skrivene stavke (.import, .git, .DS_Store...)

        var full_path_to_card := dir_path.path_join(card_name)

        if directory.current_is_dir():
            if recursive:
                all_paths_to_cards.append_array(collect_card_paths(full_path_to_c
ard, recursive))
            else:
                if card_name.ends_with(".tres") or card_name.ends_with(".res"):
                    all_paths_to_cards.append(full_path_to_card)

    directory.list_dir_end()
    return all_paths_to_cards
```

6.2.4. Razdvajanje UI-a i logike

U okviru projekta korisnički interfejs je implementiran kao tanak sloj iznad domenske logike. UI scene (paneli, trake, modalni prozori) sadrže minimalnu količinu logike: njihova glavna uloga je da osluškuju signale menadžera (*connect*), ažuriraju vidžete UI-a (poput *Label*, *ProgressBar*, *Button*) i proslede korisničke akcije nazad ka sistemu. Svi podaci o resursima, špilu, događajima i nivoima zgrada žive isključivo u autoload menadžerima dok UI ne pokušava da se meša u unutrašnju strukturu ovih podsistema. Komunikacija je *event-driven*: menadžeri emituju signale, UI ih sluša i reaguje a svo slanje akcija ide kroz jasno definisane javne metode bez direktnih *get_node()* zavisnosti u menadžerima.

Isti princip primenjuje se i na vizuelni deo. Stil i raspored UI elemenata konfigurišu se centralno preko teme *MainTheme.tres* i odgovarajućih *type variations* a ne da se stil svakog čvora ručno prepisuje kroz *theme_override_** podešavanja. Ovo zadržava konzistentan izgled interfejsa i olakšava održavanje: promena u glavnoj temi automatski se odražava na sve relevantne elemente, dok domenska logika ostaje potpuno razdvojena od toga kako je interfejs stilizovan i raspoređen.

6.3. Data driven sistem

Glavna ideja data-driven pristupa u ovoj igri jeste da sadržaj živi u resursima i da može da se menja bez rekompajliranja koda. Umesto da se balans i ponašanje igre rigidno definišu u skriptama kroz niz uslovnih grana većina važnih vrednosti (troškovi, efekti, nivoi zgrada, uslovi pojavljivanja događaja) definiše se u spoljnim resurs fajlovima.

Pri pokretanju igre baza podataka karata *CardDB* učitava sve relevantne *.tres/.res* fajlove iz *data/* direktorijuma, proverava osnovnu ispravnost podataka i izlaže ih menadžerima kao što su *DeckManager*, *EventManager* i *BuildingManager*. Na sličan način funkcioniše i *EventDB* za događaje. Menadžeri se tako oslanjaju na već pripremljen skup definicija, dok se sama pravila igre nalaze u servisnim skriptama i podsistemima koji te podatke tumače.

Na ovaj način balans igre ostaje u podacima a pravila u kodu: dodavanje nove karte, događaja ili nivoa zgrade uglavnom se svodi na kreiranje novog resurs fajla ili izmenu postojećeg, dok struktura sistema ostaje stabilna. U narednim potpoglavljima biće detaljnije prikazan model podataka za karte i događaje, kao i način na koji se efekti i troškovi kombinuju da bi se dobilo željeno ponašanje igre.

6.3.1. Šeme resursa

Centralni data-driven elementi u igri su karte i događaji. Oba tipa su modelovana kao Godot resursi, sa jasno definisanom šemom koja opisuje koje podatke nose i kako ih ostatak sistema tumači.

Karte su definisane kao resurs klase *Card*, koju deklariše skripta *Card.gd*. Svaka pojedinačna karta je jedan *.tres* fajl koji predstavlja instancu ove klase. U najkraćem, šema karte izgleda ovako:

```

@export var id: String
@export var title: String
@export var exhaust: bool = false
@export var starting_card: bool = true
@export var cost: Dictionary= {"wood" : 0, "stone" : 0, "food" : 0}
@export var effect_type: String
@export var effect_params : Dictionary = {}

const COST_ORDER := ["wood","stone","food"]

func ui_title_and_cost() -> String:
    var parts: Array[String] = []
    for k in COST_ORDER:
        var v := int(cost.get(k, 0))
        if v > 0:
            parts.append("%s-%d" % [k.capitalize(), v])

    var return_string := "%s [%s]" % [title, ", ".join(parts)] if (parts.size() > 0) else title
    return return_string

```

(A) Karta – Card.gd (Resource)

Polje *ID* predstavlja jedinstveni identifikator karte i koristi se u *CardDB* i *DeckManager* podsistemima kao ključ za pretragu i referenciranje. *title* je naslov karte koji se prikazuje u korisničkom interfejsu. Polje *exhaust* određuje da li se karta troši nakon igranja (da li odlazi trajno iz špila), dok *starting_card* označava da li karta ulazi u početni špil na nultom potezu.

Troškovi igranja karte predstavljeni su kroz rečnik *cost*, u kome su ključevi nazivi resursa (npr. "wood", "stone", "food") a vrednosti brojevi koji označavaju koliko je jedinica tog resursa potrebno da bi se karta odigrala. Polje *effect_type* označava tip efekta koji karta ima dok *effect_params* sadrži dodatne parametre potrebne za konkretan efekat. Konstanta *COST_ORDER* određuje redosled kojim će se troškovi prikazivati, a pomoćna metoda *ui_title_and_cost()* formira String koji kombinuje naslov karte i formatirane troškove, što se koristi direktno u UI sloju.

Događaji su definisani kroz resurs klasu *GameEvent*. Svaki događaj opisuje šta se dešava u igri, pod kojim uslovima se aktivira i koje efekte ima na početku, tokom trajanja

i na kraju. Pored osnovnih polja za identitet i klasifikaciju, GameEvent sadrži i niz ugnježenih resursa za uslove, efekte, ciljeve i moguće izbore:

```
enum Kind { GOOD, BAD, MIXED }
enum Tier { T1 = 1, T2 = 2, T3 = 3 }

@export var id: String
@export var title: String

@export var tier: int = Tier.T1
@export var kind: int = Kind.GOOD

@export var duration: int = 0

@export var weight: int = 1
@export var cooldown: int = 0

@export var conditions: EventConditions

@export var start_effects: Array[EventEffect] = []
@export var tick_effects: Array[EventEffect] = []
@export var end_effects: Array[EventEffect] = []

@export var choices: Array[EventChoice] = []
@export var objective: EventObjective
```

(B) Događaj – GameEvent.gd (Resource)

Polja *ID* i *title* imaju istu ulogu kao kod karata: identifikator događaja i naslov koji se prikazuje igraču. Enumeracije *Tier* i *Kind* služe za klasifikaciju događaja po složenosti (T1, T2, T3) i po „tonu“ (pozitivan, negativan ili mešoviti događaj). Polja *duration*, *weight* i *cooldown* definišu koliko dugo događaj traje, kolika mu je verovatnoća u odnosu na druge događaje da se aktivira i koliko poteza mora da prođe pre nego što ponovo može da se aktivira.

Polje *conditions* je resurs tipa *EventConditions* i opisuje minimalne uslove koje igra mora da ispuni da bi događaj uopšte bio u mogućnosti da se odigra (npr. minimalan broj poteza). Efekti događaja dele se na tri faze: *start_effects* (efekti na startu), *tick_effects* (efekti koji se primenjuju tokom svakog poteza dok je događaj aktivan) i *end_effects* (efekti pri završetku događaja). Za tier 2 događaje, polje *choices* sadrži listu mogućih

izbora (*EventChoice*), dok *objective* može da definiše cilj povezan sa događajem (npr. dostići određeni nivo resursa do zadatog poteza).

Uslovi za aktivaciju događaja grupisani su u posebnu klasu *EventConditions*. Tipično sadrže polja kao što su *min_turn* (najraniji potez na kom događaj može da se pojavi), *min_avg_building_level* (minimalan prosečan nivo zgrada), *requires_building_id* i *requires_building_min_level* (događaj zahteva da određena zgrada postoji i da je na određenom nivou). Na osnovu ovih parametara *EventManager* odlučuje da li događaj u datom trenutku ulazi u „kandidat“ skup.

```
@export var min_turn: int = 0
@export var min_avg_building_level: float = 0.0
@export var requires_building_id: String = ""
@export var requires_building_min_level: int = 0
```

(C) Uslovi

Efekti su modelovani kroz klasu *EventEffect*, koja u sebi ima polja *effect_type* i *params*. *effect_type* opisuje o kakvoj se vrsti efekta radi (na primer efekat nad resursima, zgradama ili špilom), dok *params* sadrži konkretne numeričke vrednosti ili identifikatore koji se koriste pri primeni tog efekta. Ovaj obrazac je isti i za efekte na kartama, što olakšava ponovnu upotrebu.

```
@export var effect_type: String
@export var params: Dictionary = {}
```

(D) Efekat

Ciljevi događaja opisani su klasom *EventObjective*. U okviru nje se čuvaju tip cilja (*objective_type*), dodatni parametri (*params*), kao i rok (*deadline*) što je obično broj poteza u okviru koga cilj treba da bude ispunjen. Pored toga, definisani su i efekti koji se primenjuju u slučaju uspeha (*on_success*) i neuspeha (*on_fail*), što omogućava da jedan isti događaj ima više mogućih završetaka u zavisnosti od toga kako je igrač reagovao.

```
@export var objective_type: String
@export var params: Dictionary = {}
@export var deadline: int = 0
@export var on_success: Array[EventEffect] = []
@export var on_fail: Array[EventEffect] = []
```

(E) Cilj

Na kraju izbori igrača kod tier 2 događaja opisani su kroz klasu *EventChoice*. Svaki izbor ima tekstualnu oznaku (*label*), trošak (*cost*) koji se plaća pri biranju te opcije, trajanje (*duration*) i skup efekata po fazama (*start_effects*, *tick_effects*, *end_effects*). Na ovaj način jedan događaj može da ponudi više različitih ruta, od kojih svaka ima svoje posledice i tok.

```
@export var label: String
@export var cost: Dictionary = {}
@export var duration: int = 1

@export var start_effects: Array[EventEffect] = []

@export var tick_effects: Array[EventEffect] = []

@export var end_effects: Array[EventEffect] = []
```

(F) Izbor

6.3.2. Primena efekata i troškova (servisni sloj)

Primena troškova i efekata odvija se van korisničkog interfejsa, u tzv. servisnom sloju. UI samo pokreće odgovarajuće akcije (npr. klik na kartu), dok se sama pravila poput da li karta može da se odigra, koliko košta i šta tačno radi nalaze u zasebnim funkcijama koje rade nad podsistemima kao što su *Game*, *DeckManager*, *EventManager* i *BuildingManager*.

Troškovi kartica obrađuju se preko dve funkcije. Funkcija *can_pay* čita trenutno stanje resursa iz *Game* podsistema i proverava da li igrač ima dovoljno resursa za zadati trošak. Ukoliko je uslov ispunjen poziva se funkcija *pay* koja smanjuje vrednosti resursa u *Game*-u prema zadatom rečniku i nakon toga emituje signal *resources_changed* kako bi UI i ostali delovi sistema mogli da osveže prikaz. Na ovaj način logika vezana za resurse ostaje centralizovana i nezavisna od konkretnih UI elemenata.

Efekti kartica zasnivaju se na polju *effect_type* u samoj karti. U zavisnosti od vrednosti ovog stringa sistem bira koji servis treba da obradi efekat. Na primer za *effect_type* kada je "building" poziva se metoda *BuildingManager.build_or_upgrade* koja povećava nivo odgovarajuće zgrade, emituje signal *building_changed* i, dodaje ili uklanja karte iz špila na osnovu parametara *deck_add* i *deck_remove*. Za druge tipove efekata, kao što su "resource", "draw" ili "discard", efekti se prosleđuju odgovarajućim servisima u *Game* ili

DeckManager podsistemu. Ako se naiđe na nepoznatu vrednost *effect_type* atributa sistem ispisuje upozorenje i ignoriše efekat bez rušenja toka igre. Ovo omogućava da se neispravne ili nedovršene definicije efekata bezbedno uoče i isprave.

Događaji (T1, T2, T3) koriste sličan princip, ali sa dodatnim fazama. Svaki *GameEvent* može da ima efekte u tri faze: *start_effects*, *tick_effects* i *end_effects* koji se redom primenjuju pri početku događaja, tokom svakog poteza dok je događaj aktivan i kada se događaj završi. Kod tier 2 događaja koji zahtevaju izbor igrača *EventManager* šalje signal kojim obaveštava UI da treba da se otvori modalni prozor sa listom *EventChoice* opcija. Nakon što igrač izabere jednu od opcija, sistem primenjuje trošak datog izbora (ako postoji) i izvršava njegove efekte.

Kod tier 3 događaja naglasak je na ciljevima. Svaki takav događaj ima pridruženi *EventObjective*, koji definiše tip cilja, parametre, rok do kada treba da se ispune uslovi i efekte za slučaj uspeha (*on_success*) i neuspeha (*on_fail*). *EventManager* tokom igre prati napredak u odnosu na cilj i proverava da li je cilj ispunjen ili je rok istekao i u odgovarajućem trenutku primenjuje odgovarajući skup efekata. Na ovaj način logika za troškove i posledice odluka igrača ostaje koncentrisana u servisnom sloju, dok UI služi kao sloj za prikaz i unos.

6.3.3. Primeri instanci resursa (karata i događaja)

Karta zgrade za drva prvog nivoa (Build logging camp)

```
[resource]
script = ExtResource("Card.gd")
id = "build_logging_camp"
title = "Build: Logging Camp"
exhaust = true
cost = {
  "food": 0,
  "stone": 1,
  "wood": 2
}
effect_type = "building"
effect_params = {
  "building_id": "wood_logging",
  "deck_add": [
    { "count": 3, "id": "prod_tax_logging_camp"},
    { "count": 1, "id": "upgrade_logging_cottage"}
  ],
  "deck_remove": [
    { "count": -1, "id": "gather_wood"},
    { "count": -1, "id": "build_logging_camp"}
  ],
  "to_level": 1
}
```

T2 Event- Festival (sa svojim izborima)

```
[gd_resource type="Resource" script_class="GameEvent" format=3]

[ext_resource type="Script" path="res://scripts/data/events/GameEvent.gd"
id="1"]
[ext_resource type="Script" path="res://scripts/data/events/EventEffect.gd"
id="2"]
[ext_resource type="Script" path="res://scripts/data/events/EventConditions.gd"
id="3"]
[ext_resource type="Script" path="res://scripts/data/events/EventChoice.gd"
id="4"]
```

```
##Uslovi da bi se događaj mogao pojaviti
[sub_resource type="EventConditions" id="Conditions"]
script = ExtResource("3")
min_turn = 6
min_avg_building_level = 1.0

#Efekat #1: mali globalni bonus na prinos
[sub_resource type="EventEffect" id="SmallYield"]
script = ExtResource("2")
effect_type = "yield_mod"
params = {
  "op": "add",
  "scope": "global",
  "value": 2
}

#Efekat #2: veći globalni bonus na prinos
[sub_resource type="EventEffect" id="BigYield"]
script = ExtResource("2")
effect_type = "yield_mod"
params = {
  "op": "add",
  "scope": "global",
  "value": 4
}

#Efekat #3: negativan bonus na prinos (loše posledice)
[sub_resource type="EventEffect" id="BadYield"]
script = ExtResource("2")
effect_type = "yield_mod"
params = {
  "op": "add",
  "scope": "global",
  "value": -1
}
```

```

#Izbor #1: mali festival
[sub_resource type="EventChoice" id="Choice_SmallFestival"]
script = ExtResource("4")
label = "Throw a smal festival"
tick_effects = Array[ExtResource("2")][[SubResource("SmallYield")]]

#Izbor #2: veliki festival (košta ali veći bonus i traje duže)
[sub_resource type="EventChoice" id="Choice_GrandFestival"]
script = ExtResource("4")
label = "Fund a grand festival"
cost = { "food": 3 }
duration = 2
tick_effects = [SubResource("BigYield")]

#Izbor #3: otkazivanje festivala (negativan uticaj)
[sub_resource type="EventChoice" id="Choice_Cancel_Festival"]
script = ExtResource("4")
label = "Cancel the festival"
tick_effects = SubResource("BadYield")]

#Glavni resurs događaja
[resource]
script = ExtResource("1")
id = "t2_festival"
title = "Festival"
tier = 2
kind = 2
weight = 6
cooldown = 8
conditions = SubResource("Conditions")
choices =[SubResource("Choice_SmallFestival"),
          SubResource("Choice_GrandFestival"),
          SubResource("Choice_CancelFestival")
]

```

T3 event – Royal Decree (sa svojim uslovima, ciljem i nagradom)

```
[gd_resource type="Resource" script_class="GameEvent" load_steps=11 format=3
uid="uid://bqkxfdkfced3b"]

[ext_resource type="Script" uid="uid://icwe5ip32rqs"
path="res://scripts/data/events/EventChoice.gd" id="1_v6p1k"]
[ext_resource type="Script" uid="uid://b21nv017n6scu"
path="res://scripts/data/events/EventEffect.gd" id="2_2o7iq"]
[ext_resource type="Script" uid="uid://c1n5nv13fsfbd"
path="res://scripts/data/events/EventConditions.gd" id="2_02t04"]
[ext_resource type="Script" uid="uid://cy4k8s2pd1rd1"
path="res://scripts/data/events/GameEvent.gd" id="3_02t04"]
[ext_resource type="Script" uid="uid://dgojnetuurdjd"
path="res://scripts/data/events/EventObjective.gd" id="4_hdb2n"]

[sub_resource type="Resource" id="Resource_ajjie"]
script = ExtResource("2_02t04")
min_avg_building_level = 2.0
requires_building_id = "wood_logging"
requires_building_min_level = 1
metadata/_custom_type_script = "uid://c1n5nv13fsfbd"

[sub_resource type="Resource" id="Resource_et5uj"]
script = ExtResource("2_2o7iq")
effect_type = "gain_resource"
params = {
"amount": -5,
"resource": "wood"
}
metadata/_custom_type_script = "uid://b21nv017n6scu"

[sub_resource type="Resource" id="Resource_u6yhj"]
script = ExtResource("2_2o7iq")
effect_type = "building_upgrade"
params = {
"building_id": "wood_logging",
"to_level": 3
}
metadata/_custom_type_script = "uid://b21nv017n6scu"

[sub_resource type="Resource" id="Resource_tfqoc"]
script = ExtResource("2_2o7iq")
effect_type = "gain_resource"
params = {
"amount": 2,
```

```

"resource": "stone"
}
metadata/_custom_type_script = "uid://b21nv017n6scu"

[sub_resource type="Resource" id="Resource_5y4ee"]
script = ExtResource("4_hdb2n")
objective_type = "play_cards_gain_resource"
params = {
  "min_plays": 3,
  "resource": "wood"
}
deadline = 2
on_success = Array[ExtResource("2_2o7iq")][[SubResource("Resource_u6yhj"),
SubResource("Resource_tfqoc")]]
on_fail = Array[ExtResource("2_2o7iq")][[SubResource("Resource_et5uj")]]
metadata/_custom_type_script = "uid://dgojnetuurdjd"

[resource]
script = ExtResource("3_02t04")
id = "t3_royal_decree_wood"
title = "Royal Decree: Wood"
tier = 3
kind = 2
weight = 4
cooldown = 10
conditions = SubResource("Resource_ajiie")
objective = SubResource("Resource_5y4ee")
metadata/_custom_type_script = "uid://cy4k8s2pd1rd1"

```

6.3.4. Prednosti i posledice ovakvog pristupa

Ovakav data-driven pristup omogućava da se balans igre menja bez izmene koda. Troškovi, efekti, ciljevi i uslovi događaja definišu se u *.tres* fajlovima pa je često dovoljno samo prilagoditi vrednosti u resursima da bi se promenila težina igre, nagrade ili posledice određenih odluka. To u praksi znači da se iteracije na balansu mogu raditi brzo, bez ulaska u servisni sloj i menadžere.

Druga važna posledica je da je proširenje sadržaja relativno jednostavno. Dodavanje nove karte ili događaja svodi se na kreiranje nove instance odgovarajućeg resursa (na osnovu šema opisanih u prethodnim potpoglavljima), dok menadžeri poput *DeckManager-a* i *EventManager-a* nastavljaju da rade sa istim skupom pravila. Samim

tim, projekat je spreman da podrži veći broj karata i događaja bez potrebe za stalnim širenjem postojeće logike.

Konačno, kombinacija jedinstvenog *seed*-a i jasno odvojenih servisnih slojeva čini sistem pogodnim za testiranje i reprodukciju problema. Pošto su efekti i troškovi primenjeni van UI-a a nasumične odluke zavise od poznatog *seed*-a za oponašanje odigrane sesije često je dovoljno zapamtiti *seed* i niz igračevih akcija. Ovo olakšava i ručno testiranje i eventualnu automatizaciju scenarija u budućim verzijama projekta.

7. Rezultati i evaluacija

Cilj ovog poglavlja je da prikaže šta je tokom izrade rada zaista implementirano i kako se ponaša u praksi, a zatim da sumira kvalitativnu evaluaciju, evidentira ograničenja i predloži sledeće korake. Fokus je na dokazivanju da je jezgro petlje (karte, efekti, resursi/zgrade, događaji) funkcionalno, deterministično i da ima mogućnosti za lakim širenjem.

7.1. Funkcionalni rezultati igre „Karte i kolonije“

U tabeli su prikazani funkcionalni rezultati koji su postignuti prilikom izrade ovog rada.

Oblast	Rezultat (status)	Evidencija / Napomena
Jezgro špila	Refill ruke, odigravanje karata, trošak efekti, discard/exhaust	DeckManager emituje hand_changed; trošak ide preko Game resursa; efekti su data-driven
Resursi	Praćenje i ažuriranje	Game.resources_changed ažurira gornji bar; format troška u UI ("Title [Wood-2, Stone-1]").
Zgrade	Nivoi po building_id, signal building_changed	BuildingManager održava levels; leva lista se automatski osvežava.
Događaji T1/T2/T3	Faze i izbori	T2 modal radi (emit → UI → apply choice); T3 ciljevi prikazani na desnom panelu; tick/start/end efekti su modelovani kroz EventEffect.
Tema i UI	Konzistentan layout, procentualni raspored, centralna tema	Gornji bar, levi/desni sidebar i donji Hand bar su responzivni; karte centrirane; overflow rešen (wrap/label).
Determinizam	Seed-ovan tok	Jedinstven RNG seed u Game; isti ulazi ⇒ isti ishod (korisno za reprodukciju).
Arhitektura	Modularno, signalima povezano	Autoload menadžeri bez UI zavisnosti; UI „tanak“, reaguje na signale.

Kao što se može videti iz tabele implementirani su i demonstrirani jezgro špila, rukovanje resursima i zgradama i osnovna arhitektura za determinizam u okviru igre.

7.2. Tehničko ponašanje i UX

Tokom testiranja igre nije primećeno značajno usporavanje rada sistema. Pošto se korisnički interfejs oslanja isključivo na signale za osvežavanje izbegava se nepotrebno stalno renderovanje i dodatni pritisak na garbage collector. U praksi to znači da i pri brzem igranju, kada se odigra više karata u kratkom vremenu, igra ostaje aktivna i ne dolazi do primetnih štucanja u prikazu.

Sa stanovišta stabilnosti tok poteza, od vučenja karata, preko njihovog odigravanja i primene efekata, do obračuna resursa i stanja zgrada, prolazi bez runtime grešaka. Sloj za bazu podataka (*CardDB/EventDB*) pri učitavanju resursa obavlja osnovne validacije i, u slučaju problema (na primer dupliran id ili nepoznat *effect_type*), upisuje upozorenja u log. Na taj način potencijalne greške u konfiguraciji podataka ne ruše tok igre, ali ostaju vidljive za dalju analizu.

Što se korisničkog iskustva tiče raspored elemenata interfejsa ostaje stabilan i pri promeni režima rada odnosno dimenzija. Kartice u ruci formatirane su dosledno, tekst na kartama koristi prelom reda (wrap), tako da duži naslovi i opisi ne razbijaju raspored. Bočni paneli (sidebar-i) za zgrade i ciljeve ostaju čitljivi i tematski usklađeni sa ostatkom UI-ja, što doprinosi tome da igrač u svakom trenutku može jasno da vidi trenutno stanje resursa, špila i aktivnih događaja.

7.3. Ostvarenje ciljeva dizajna

Cilj (iz dizajna)	Ocena ostvarenja	Komentar
Data-driven sadržaj (karte/događaji)	Ispunjeno (POC)	Kartice i događaji su .tres resursi; balans i tok se menjaju bez koda.
Modularna arhitektura	Ispunjeno (POC)	Menadžeri (Game/Deck/Event/Building) + UI sloj; komunikacija signalima.
Determinizam (seed)	Ispunjeno (POC)	Ponovljive sesije, olakšano debugovanje.
Čitljiv UI	Uglavnom ispunjeno	Tema i raspored su stabilni; ostaje poliranje vizuelnih komponenti i stanja kontrola.
Mogućnost proširenja sadržaja sadržaja	Ispunjeno (POC)	Dodavanje novih karata/događaja bez izmene koda je demonstrirano.

7.4. Poznata ograničenja trenutne verzije

Iako jezgro sistema radi stabilno i demonstrira osnovne mehanike, trenutna verzija ima više ograničenja od kojih su najvažnija sledeća:

- Sadržaj i balans. Postoji mali broj karata i događaja i kriva težine nije sistematski podešena. Balans je urađen samo okvirno, bez ozbiljnijeg testiranja i finog podešavanja.
- Meta-progres i „retention“ petlje. Ne postoje trajne nadogradnje, otključavanja novih opcija niti sinergije „višeg reda“ koje bi podstakle dugoročnije igranje i vraćanje igrača.
- Save/Load i telemetrija. Trenutna verzija nema sistem za čuvanje i učitavanje stanja partije, niti prikuplja statistike o igranju (trajanje partije, učestalost događaja, procenti pobede i sl.).
- Uvod i obuka. Nedostaje interaktivni onboarding i kontekstualni hintovi koji bi novim igračima jasno objasnili osnovne mehanike i tok partije.

- Pristupačnost i UX detalji. Ne postoje opcije za podešavanje veličine fonta, alternativne palete za color-blind igrače, kao ni podrška za navigaciju tastaturom ili kontrolerom.
- Automatizovani testovi. Formalno „test“ poglavlje nije realizovano u kodu; ne postoje unit ili integracioni testovi, pa se verifikacija ponašanja oslanja na ručno testiranje.

8. Zaključak

Ovaj rad je pokazao da je moguće osmisлити i implementirati prototip (POC) hibridne 2D igre koja kombinuje tri žanra: city-builder, deck-builder i roguelike, na data-driven temeljima i sa jasno razdvojenim odgovornostima sistema. Kroz arhitekturu zasnovanu na autoloading menadžerima (Game, DeckManager, EventManager, BuildingManager, CardDB), UI sloju koji se oslanja na signale, kao i tematski ujednačen i responzivan interfejs, demonstrirana je izvodljivost gameplay loop-a: karte, efekti, resursi/zgrade, događaji. Prototip je determinističan (seed-ovan RNG), što olakšava naknadno testiranje.

Sa tržišne strane, uvođenjem jasnih metrika i ograničavanjem na Steam/SteamDB podatke, izvedena je metodološki konzistentna analiza tri reprezentativna naslova. SWOT prikaz je iz tih brojeva i kvalitativnih uvida izveo praktične smernice: potrebu za prepoznatljivim identitetom, pristupačnim onboarding-om uz dovoljnu dubinu i modularnost sistema karata. Ove smernice su direktno pretočene u tehničke odluke: od data-driven modela do načina grananja efekata i izolacije UI-a od domenske logike.

8.1. Naučene lekcije

- Rano uvođenje teme i tip-varijacija smanjuje kasnije troškove UI poliranja.
- Signalni (event-driven) model smanjuje spregu, olakšava izolovano pokretanje podsistema i ubrzava iteraciju.
- Data-driven pristup realno ubrzava balansiranje i dodavanje sadržaja, ali traži disciplinu u validaciji (tipovi, ključevi, „Variant“ upozorenja).
- Jedan izvor istine za metrike (SteamDB) sprečava „mešanje jabuka i krušaka“ u tržišnim poređenjima.

8.2. Preporuke i sledeći koraci

- Sadržaj & balans: proširiti set karata/događaja (20–30/10–15), uvesti tier-ove i osnovne sinergije; iterativno kalibrisati troškove/efekte.
- Meta-progres: jednostavan sistem otključavanja posle run-a (trajni benefiti, varijante špila).
- Onboarding & UX: vođeni prvi run, kontekstualni hintovi; osnovne pristupačne opcije.

- Telemetrija & QA: logovi odigranih karata, trajanje poteza/run-a; smoke/regresioni scenariji sa fiksnim seed-om.
- Save/Load: snapshot ključnih stanja (Game/Deck/Events/Buildings).
- Vizuelno poliranje: jedinstveni states za kontrole, diskretne animacije, audio stub.

8.3. Završna reč

Projekat je ispunio primarni cilj: potvrdio je izvodljivost hibridne igre na jasno definisanim tehničkim temeljima, uz metodološki čistu tržišnu analizu i operacionalizaciju SWOT uvida u konkretne dizajn-odluke. Iako je POC svesno ograničen po obimu sadržaja, arhitektura, data-driven model i deterministička petlja čine stabilnu osnovu za dalji rast — i sa razvojne i sa tržišne strane. Sledeći inkrementi mogu se realizovati bez promene arhitekture, fokusom na sadržaj, balans i UX, što je najbolji signal da je temelj postavljen ispravno.

9. Literatura

Cities: Skylines. (2015). Cities: Skylines – Steam store page. Valve / Steam. Preuzeto u junu 2025. sa

https://store.steampowered.com/app/255710/Cities_Skylines/

Cureus. (2025). From controller to screen: A narrative review of video games in medicine.

Cureus. Preuzeto 7. novembra 2025. sa

<https://www.cureus.com/articles/353586-from-controller-to-screen-a-narrative-review-of-video-games-in-medicine#!>

Dentsu. (2024, 22. oktobar). State of Gaming report: Gaming industry worth £141.7bn, surpasses music and film combined. Marketing Beat / GWI. Preuzeto 7. novembra 2025.

sa

<https://www.marketing-beat.co.uk/2024/10/22/dentsu-gaming-data-new/>

Dead Cells. (2018). Dead Cells – Steam store page. Motion Twin. Preuzeto u junu 2025.

sa

https://store.steampowered.com/app/588650/Dead_Cells/

Dead Cells review – one of the best roguelikes to date. (n.d.). Game Informer. Preuzeto

7. novembra 2025. sa

<https://www.gameinformer.com/dead-cells-review>

Esposito, O. (2005). A short and simple definition of what a videogame is. U Proceedings of DiGRA. Preuzeto 7. novembra 2025. sa

<https://www.utc.fr/~nesposit/publications/esposito2005definition.pdf>

Filament Games. (2023). Game-based learning for medical training. Filament Games

Blog. Preuzeto 7. novembra 2025. sa

<https://www.filamentgames.com/blog/game-based-learning-for-medical-training/>

Forbes Agency Council. (2023, 17. novembar). The gaming industry: A behemoth with unprecedented global reach. Forbes. Preuzeto 7. novembra 2025. sa

<https://www.forbes.com/councils/forbesagencycouncil/2023/11/17/the-gaming-industry-a-behemoth-with-unprecedented-global-reach/>

Graham, L. (n.d.). Role of video games in improving health-related outcomes. PMC.

Preuzeto 7. novembra 2025. sa

<https://pmc.ncbi.nlm.nih.gov/articles/PMC3391574/>

- Guardiola, E. (2016). The Gameplay Loop: A Player Activity Model for Game Design and Analysis. U ACE '16: Proceedings of the 13th International Conference on Advances in Computer Entertainment Technology. ACM.
<https://doi.org/10.1145/3001773.3001791>
- Jackson, M., & Odeh, H. (2025). Game-based educational experience in clinical simulation and assessment. BMC Medical Education. Preuzeto 7. novembra 2025. sa
<https://bmcmmededuc.biomedcentral.com/articles/10.1186/s12909-025-07070-5>
- Jonah's Books & Rants. (2023, novembar). The addictive loop of Slay the Spire. Medium. Preuzeto 7. novembra 2025. sa
<https://jonahsbooks.medium.com>
- Lambert, J. (2015, 31. maj). How the video game industry is changing. Investopedia. Preuzeto 7. novembra 2025. sa
<https://www.investopedia.com/articles/investing/053115/how-video-game-industry-changing.asp>
- Sicart, M. (2008). Defining game mechanics. Game Studies, 8(2). Preuzeto 7. novembra 2025. sa
<https://gamestudies.org/0802/articles/sicart>
- Slay the Spire. (2019). Slay the Spire – Steam store page. Mega Crit Games. Preuzeto u junu 2025. sa
https://store.steampowered.com/app/646570/Slay_the_Spire/
- Smiderle, R., Rigo, S. J., Marques, L. B., Coelho, J. A. P. d. M., & Jaques, P. A. (2020). The impact of gamification on students' learning, engagement and behavior based on personality traits. Smart Learning Environments, 7, 3. Preuzeto 7. novembra 2025. sa
<https://slejournal.springeropen.com/articles/10.1186/s40561-019-0098-x>
- SteamDB. (n.d.). Free database of Steam games and statistics. Preuzeto 7. novembra 2025. sa
<https://steamdb.info/>